

Practical 4 - Optimal fitting and forecasting

Practical 4a: Optimal fitting and forecasting

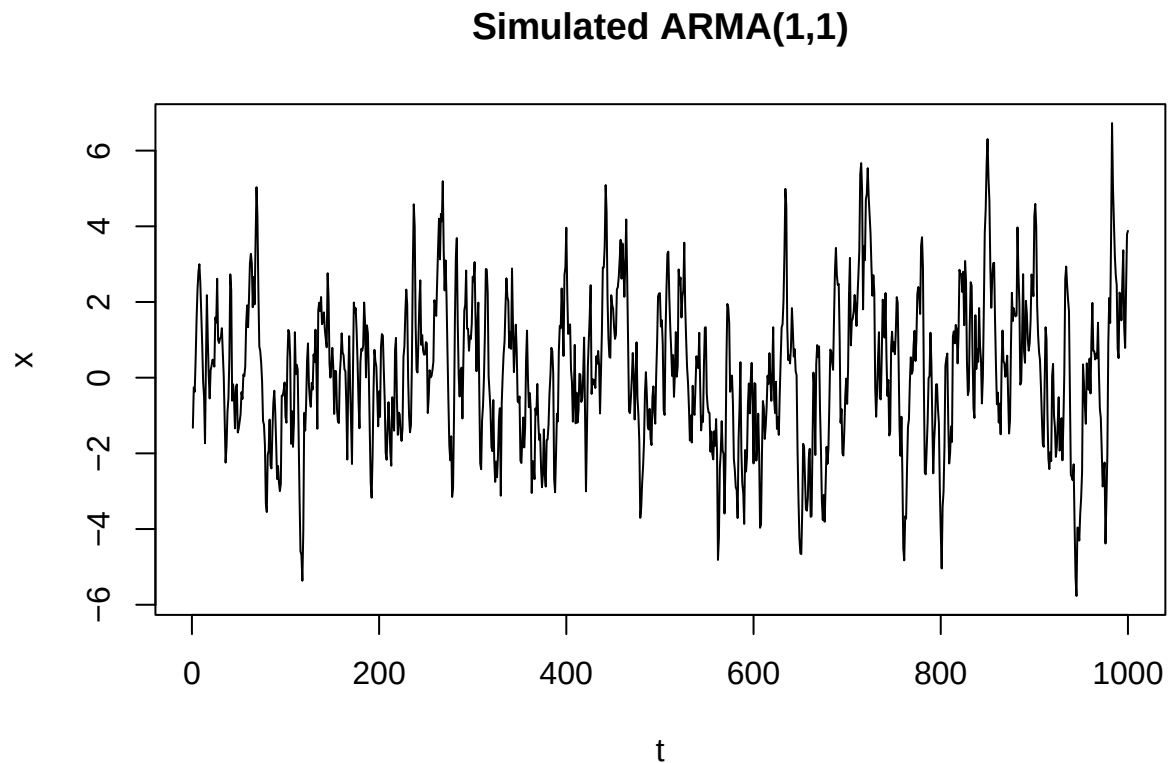
A) Choose p and 1 and simulate an ARMA(p,q) model

We simulate an ARMA(1, 1) process and compare it with the automatically selected ARIMA model.

```
set.seed(123)

n <- 1000
x <- arima.sim(model = list(order=c(1, 0, 1), ar = 0.8, ma = 0.3), n = n)

plot(x, type="l", main="Simulated ARMA(1,1)", ylab="x", xlab="t")
```



Type in:

```
y <- auto.arima(x)
y
```

```
## Series: x
## ARIMA(1,0,1) with zero mean
##
## Coefficients:
##          ar1      ma1
##      0.7782  0.3008
## s.e.  0.0227  0.0342
##
## sigma^2 = 1.006: log likelihood = -1421.63
## AIC=2849.26   AICc=2849.28   BIC=2863.98
```

With automatic ARIMA fitting we get:

ARIMA(1,0,1) with zero mean

Coefficients: ar1 ma1 0.7782 0.3008 s.e. 0.0227 0.0342

Estimated coefficients AR and MA are close to the true values used for simulation (0.8 and 0.3 respectively), not completely exact due to randomness.

Try:

```
y <- auto.arima(x, max.p=2, max.q=2)
y
```

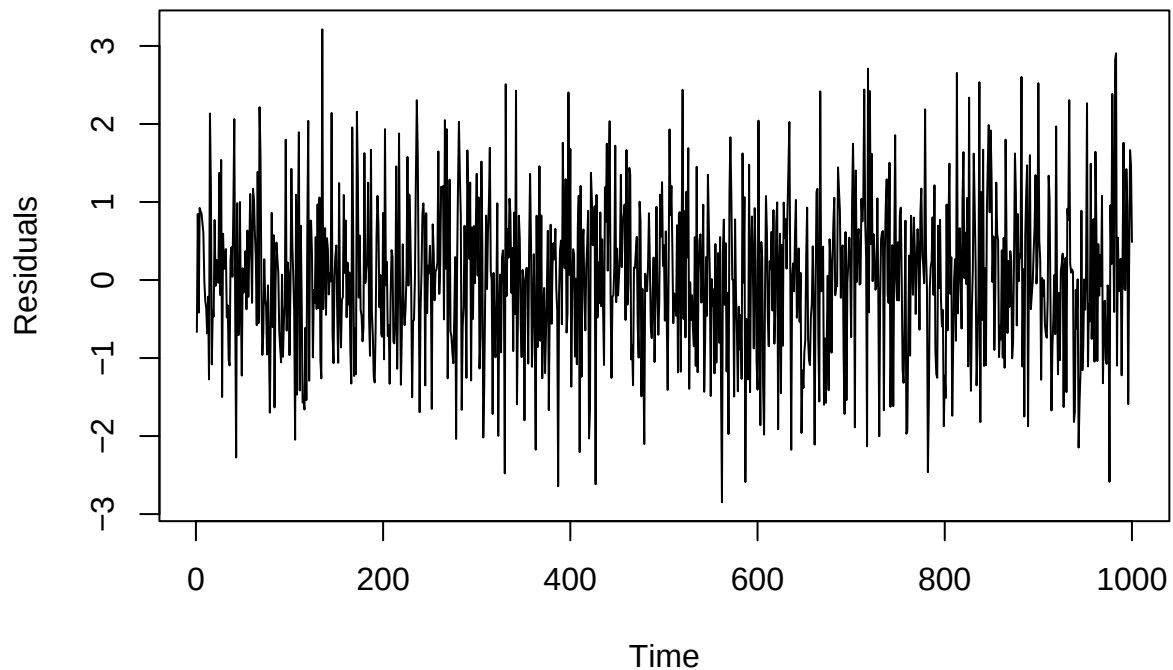
```
## Series: x
## ARIMA(1,0,1) with zero mean
##
## Coefficients:
##          ar1      ma1
##      0.7782  0.3008
## s.e.  0.0227  0.0342
##
## sigma^2 = 1.006: log likelihood = -1421.63
## AIC=2849.26   AICc=2849.28   BIC=2863.98
```

With max.p=2 and max.q=2, we get the same model as before, ARIMA(1,0,1).

Compute the residual analysis to check whether the fitted model is adequate and check that z is an IID noise:

```
res <- resid(y)
plot(res, type="l", main="Residuals of fitted ARIMA(1,0,1)", ylab="Residuals", xlab="Time")
```

Residuals of fitted ARIMA(1,0,1)



```
h <- round(log(n))  
Box.test(res, lag=h, type="Ljung-Box")
```

```
##  
## Box-Ljung test  
##  
## data: res  
## X-squared = 1.7931, df = 7, p-value = 0.9704
```

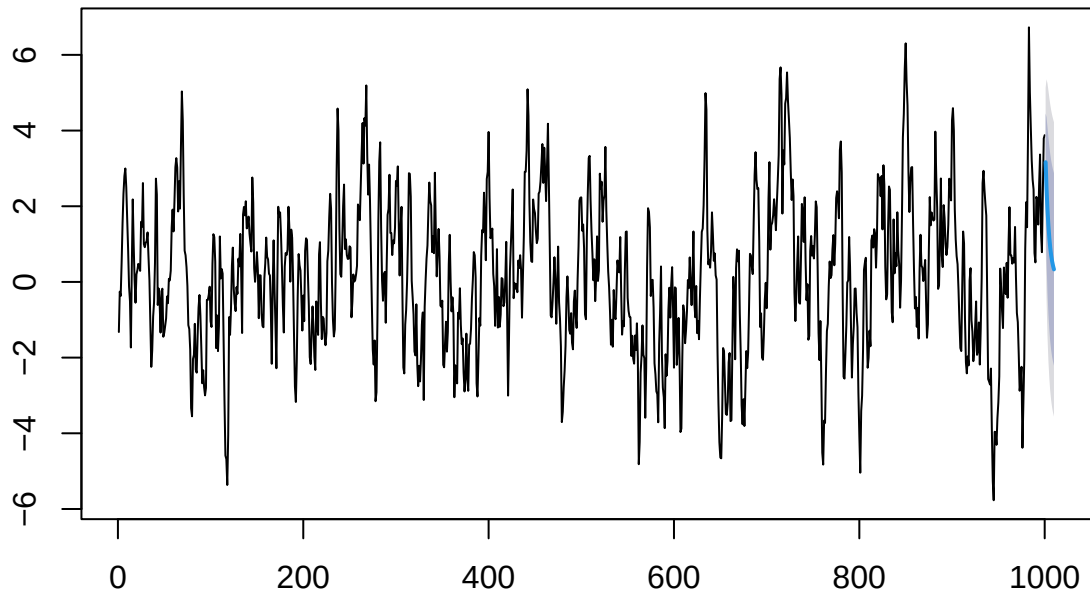
Observations:

We limit model search to $p \leq 2$, $q \leq 2$ and $d = 0$. Auto.arima chooses the best ARIMA model, and the residuals are white noise. We can see this with: - Plot: there is no visible pattern, no trend and constant variance. - Ljung-Box test: we obtain a p-value of 0.9704, which is much larger than 0.05, so we do not reject the null hypothesis of no autocorrelation in the residuals. This confirms that the fitted ARIMA(1,0,1) model is adequate for the simulated data.

To forecast we use the instruction:

```
xfc <- forecast(y)  
plot(xfc, main="Forecast ARMA(1,1)")
```

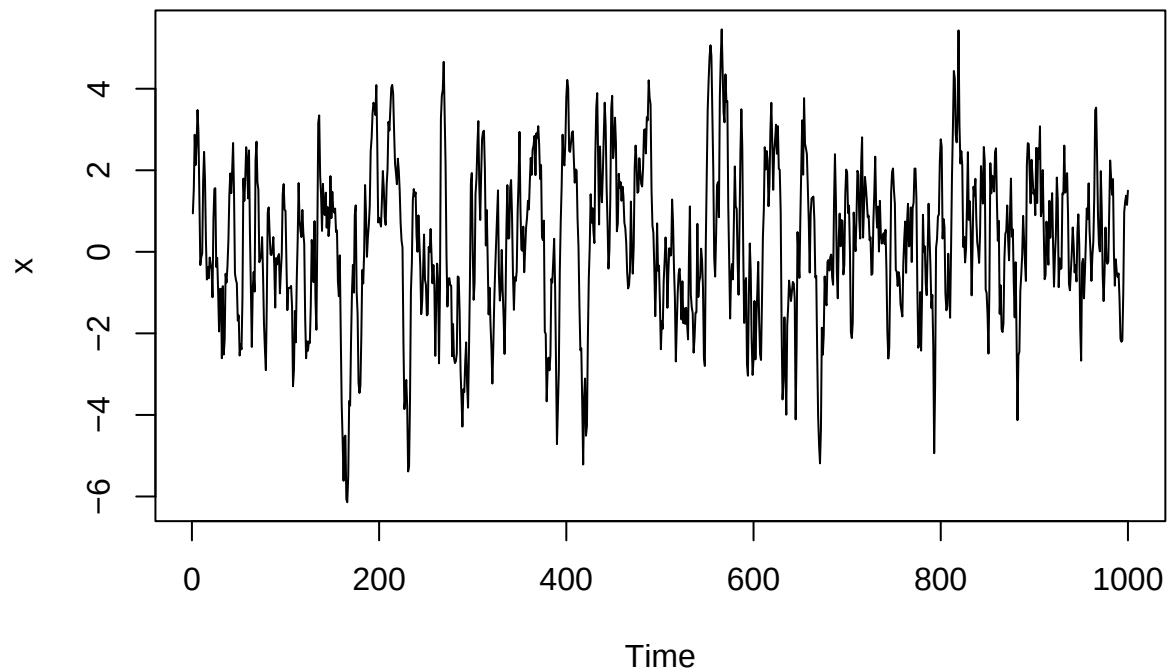
Forecast ARMA(1,1)



Note: We should forecast the model, not the raw data. Typo in the practice.

Example:

```
n <- 1000
x <- arima.sim(list(order=c(1, 0, 1), ar=(0.8), ma=c(0.3)), n = n)
plot(x, type="l")
```



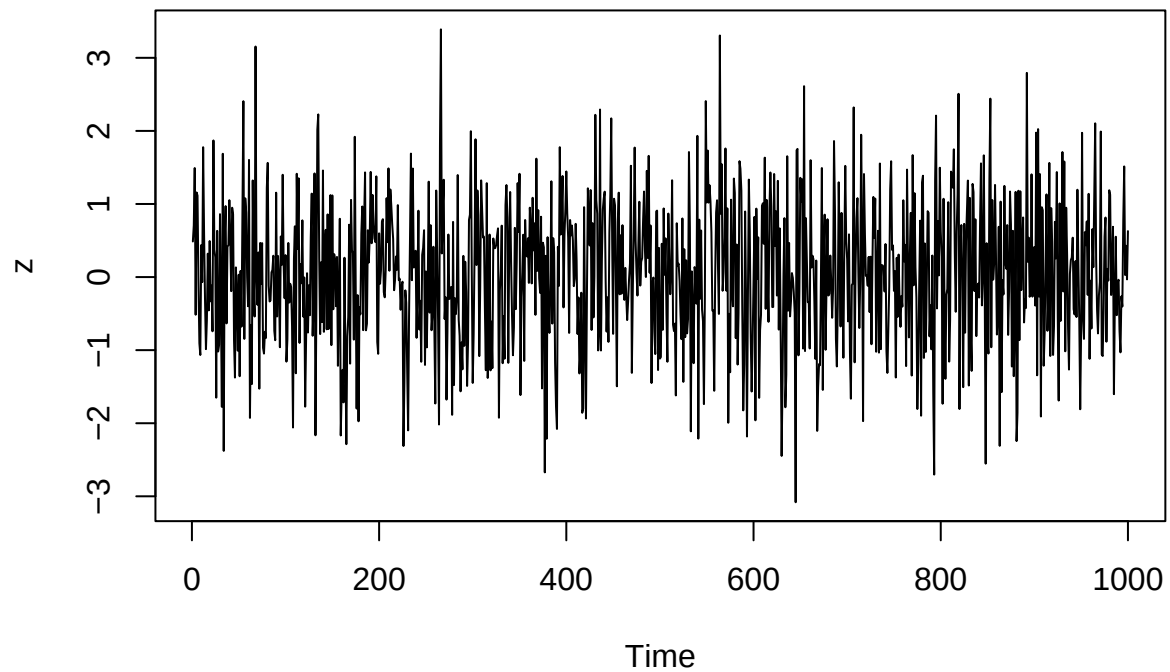
```
Box.test(x, lag=log(n), type=c("Ljung-Box"))
```

```
##
## Box-Ljung test
##
## data: x
## X-squared = 1652.9, df = 6, p-value < 2.2e-16
```

```
y <- auto.arima(x)
y
```

```
## Series: x
## ARIMA(1,0,1) with zero mean
##
## Coefficients:
##          ar1      ma1
##          0.7661  0.2985
## s.e.  0.0236  0.0362
##
## sigma^2 = 0.9928: log likelihood = -1415.04
## AIC=2836.08  AICc=2836.11  BIC=2850.81
```

```
z <- resid(y)
plot(z)
```

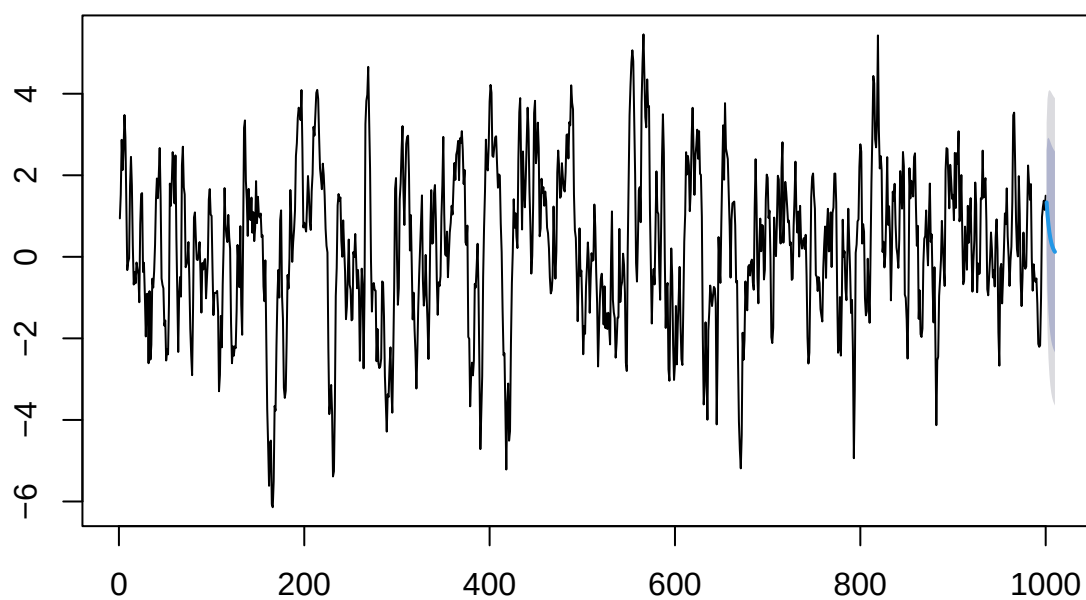


```
h <- log(n)
Box.test(z, lag=h, type=c("Ljung-Box"))
```

```
##
## Box-Ljung test
##
## data: z
## X-squared = 1.8908, df = 6, p-value = 0.9295
```

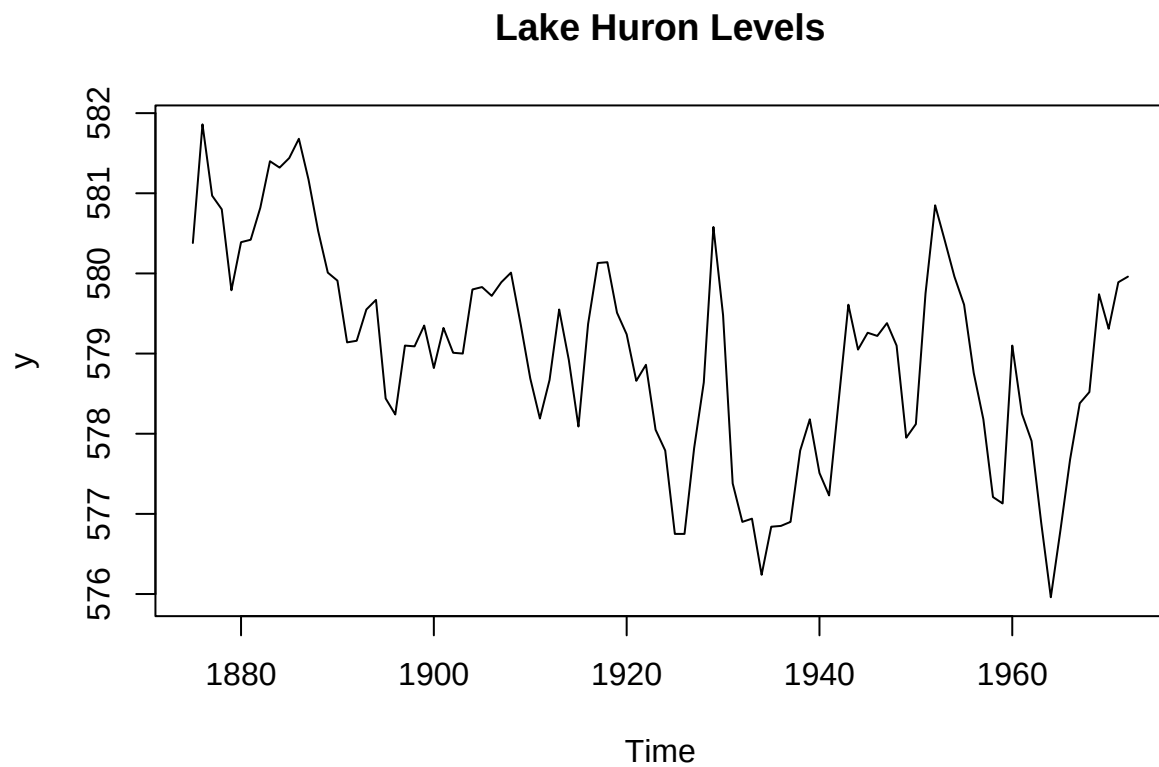
```
xfc <- forecast(y)
plot(xfc, type="l")
```

Forecasts from ARIMA(1,0,1) with zero mean



Do the same with real data:

```
y <- LakeHuron  
plot(y, type="l", main="Lake Huron Levels")
```



Fit an ARIMA model to the data:

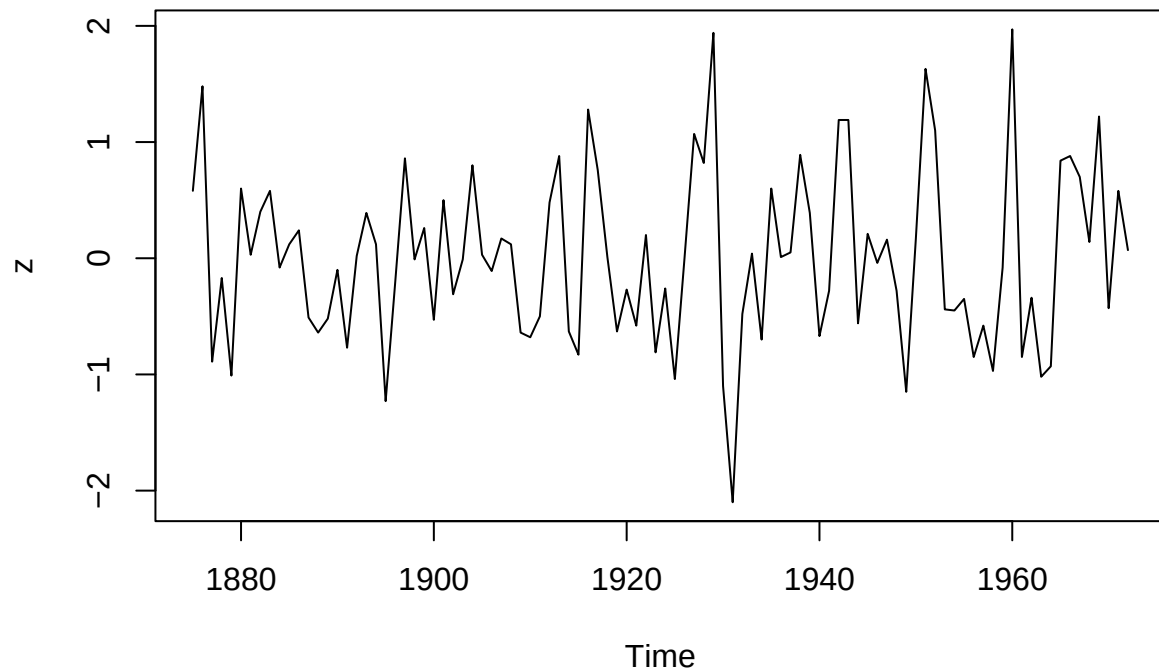
```
lh <- auto.arima(y)
lh
```

```
## Series: y
## ARIMA(0,1,0)
##
## sigma^2 = 0.5588: log likelihood = -109.11
## AIC=220.22   AICc=220.26   BIC=222.79
```

Residuals:

```
z <- resid(lh)
plot(z, type="l", main="Residuals (Lake Huron)")
```


Residuals (Lake Huron)



Check IID noise:

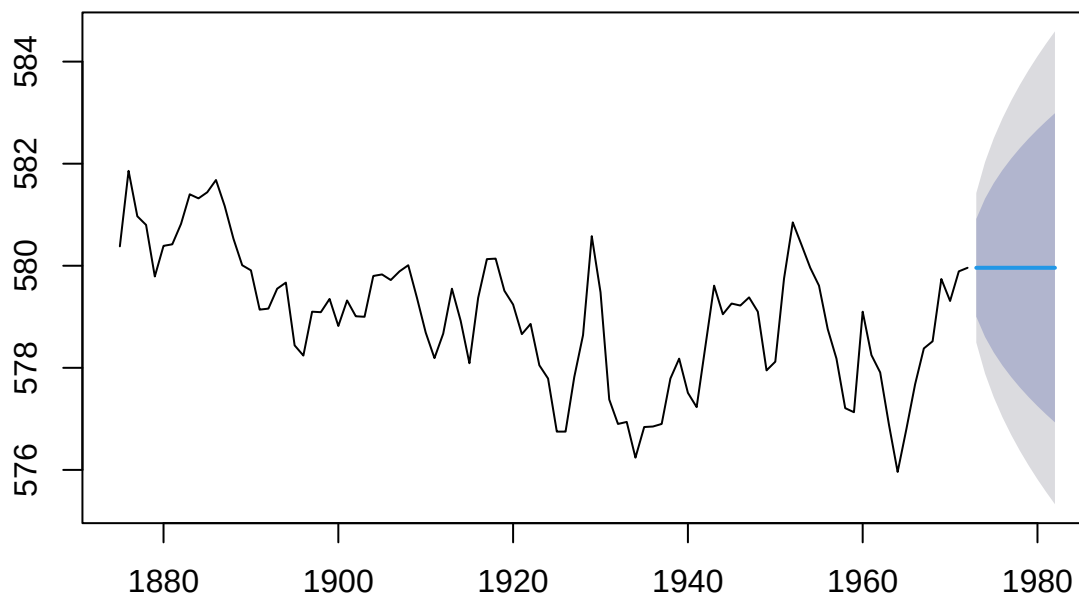
```
h <- log(length(y))
Box.test(z, lag=h, type="Ljung-Box")

##
## Box-Ljung test
##
## data: z
## X-squared = 11.342, df = 4, p-value = 0.02297
```

To forecast do:

```
lhfc <- forecast(lh)
plot(lhfc, type="l", main="Forecast Lake Huron")
```

Forecast Lake Huron



We see in this first iteration that the model found is: - ARIMA(0, 1, 0): this is a random walk model. The residual test gives a p-value of 0.02297, rejecting the IID assumption. Residuals are not white noise.

This means that the model selected by `auto.arima()` is not adequate. Leaves structure in the residuals and fails the Ljung-Box test.

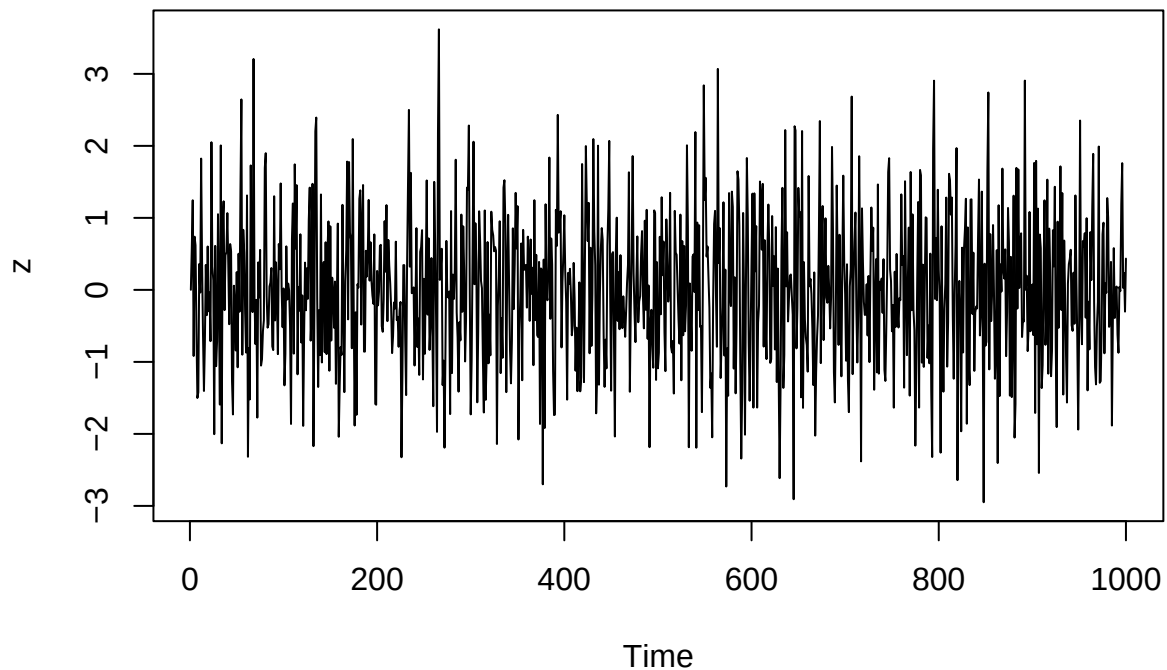
Next example improves the randomness of the residuals:

```
lh <- arima(x, order=c(1, 1, 1), method="ML")
lh

##
## Call:
## arima(x = x, order = c(1, 1, 1), method = "ML")
##
## Coefficients:
##          ar1      ma1
##       -0.4009  0.5789
## s.e.    0.1038  0.0912
##
## sigma^2 estimated as 1.098:  log likelihood = -1464.27,  aic = 2934.53
```

Now the model includes AR(1) component, MA(1) component and differencing (d=1).

```
z <- resid(lh)
plot(z, type="l")
```



We see more random behaviour than the previous example, with less structure.

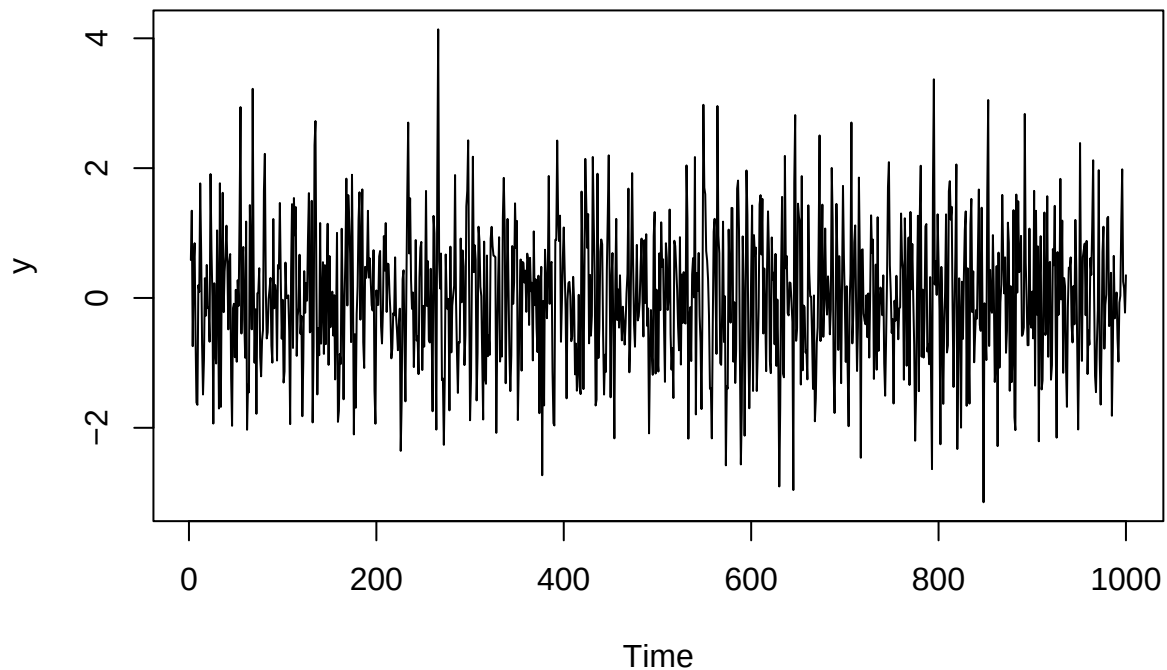
```
h <- log(98)
Box.test(z, lag=h, type="Ljung-Box")

##
## Box-Ljung test
##
## data: z
## X-squared = 17.437, df = 4, p-value = 0.001589
```

We obtain a p-value of 0.0016 which is smaller than before. The ARIMA(1, 1, 1) model does not improve the fit. The Ljung-Box test returns a very small p-value (0.0016) indicating that the residuals are not white noise. The model performs worst than a simpler ARIMA(0, 1, 0) model. Specially with limited data (LakeHuron has aprox 98 points) there is not enough data to estimate complex models. Adding AR + MA introduces noise instead of structure.

Another version:

```
y <- diff(x)
plot(y)
```



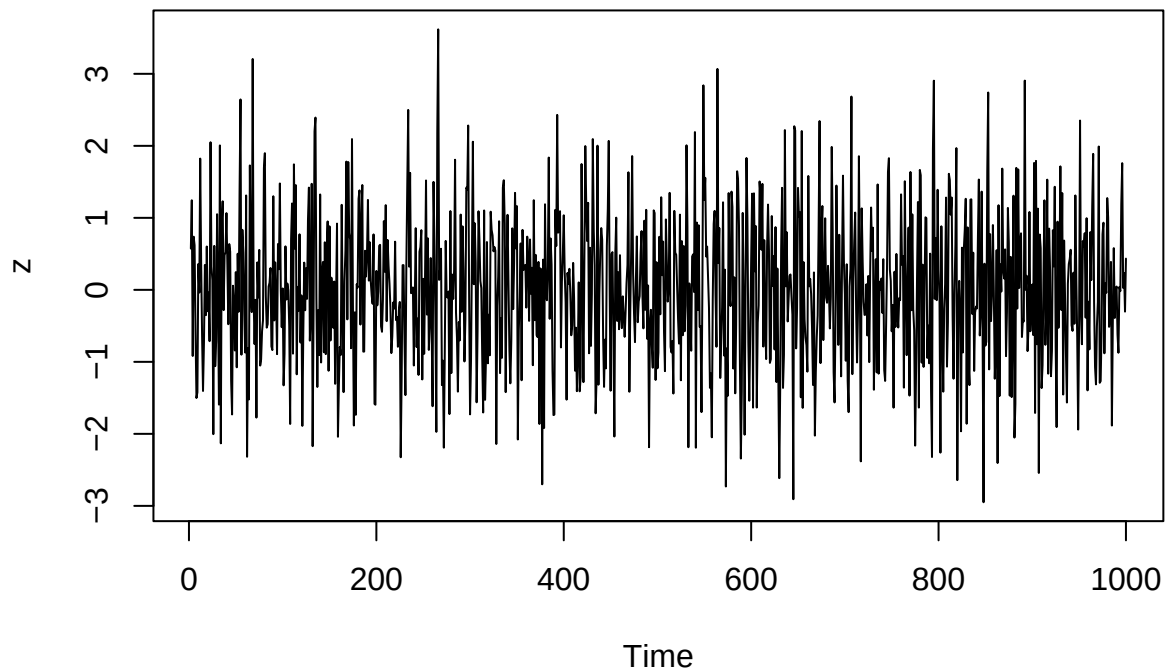
Instead of fitting ARIMA(1, 1, 1) directly on the original series, we can differentiate it first. This removes trend/non-stationarity and makes the series more stable.

```
lh <- arima(y, order=c(1, 0, 1), method="ML")
lh

##
## Call:
## arima(x = y, order = c(1, 0, 1), method = "ML")
##
## Coefficients:
##          ar1      ma1  intercept
##       -0.4009  0.5789    0.0006
## s.e.   0.1038  0.0912    0.0374
##
## sigma^2 estimated as 1.098:  log likelihood = -1464.27,  aic = 2936.53
```

It has the same structure as before but now we have applied it to a stationary series.

```
z <- resid(lh)
plot(z, type="l")
```



We observe more randomness than before.

```
h <- log(98)
Box.test(z, lag=h, type="Ljung-Box")
```

```
##
## Box-Ljung test
##
## data: z
## X-squared = 17.42, df = 4, p-value = 0.001602
```

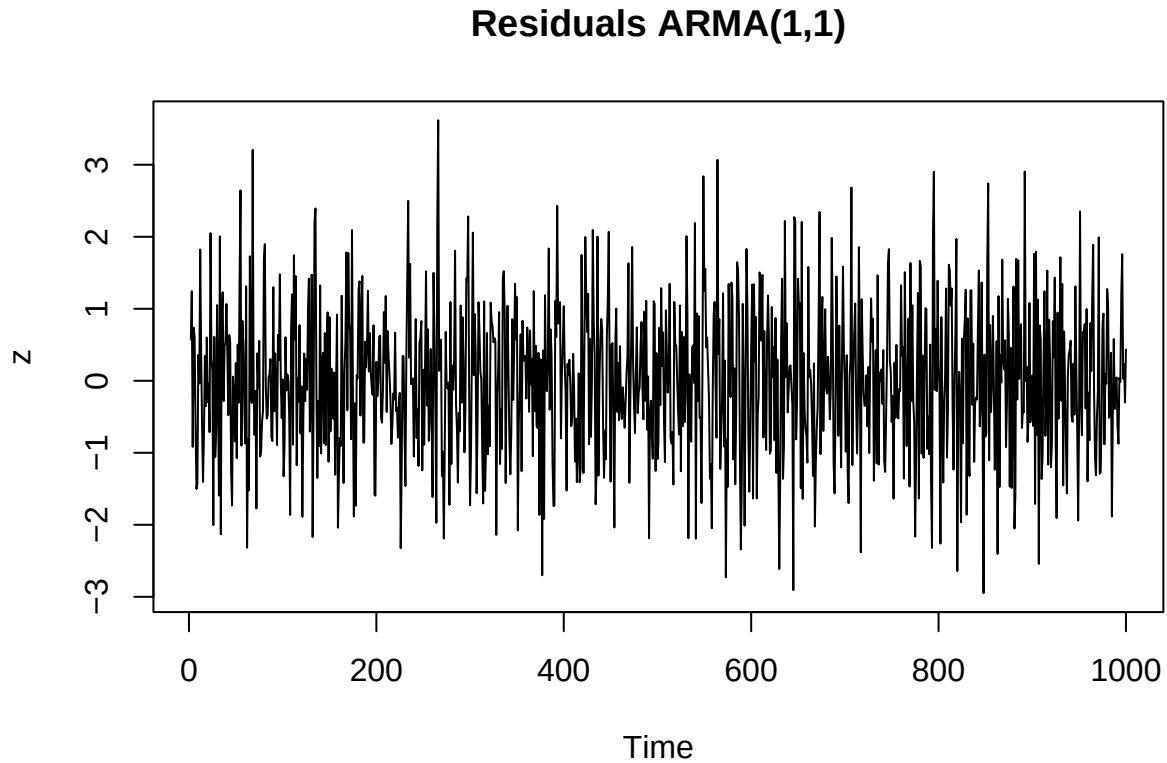
Fit ARMA(1, 1)

```
lh <- arima(y, order=c(1,0,1), method="ML")
lh
```

```
##
## Call:
## arima(x = y, order = c(1, 0, 1), method = "ML")
##
## Coefficients:
##          ar1      ma1  intercept
##        -0.4009  0.5789    0.0006
## s.e.    0.1038  0.0912    0.0374
##
## sigma^2 estimated as 1.098: log likelihood = -1464.27, aic = 2936.53
```

We see that ARM and MA coefficients are estimated. Model assumes short memory dependence.

```
z <- resid(lh)
plot(z, type="l", main="Residuals ARMA(1,1)")
```



Residuals look more random than previous models. There's no obvious trend or structure.

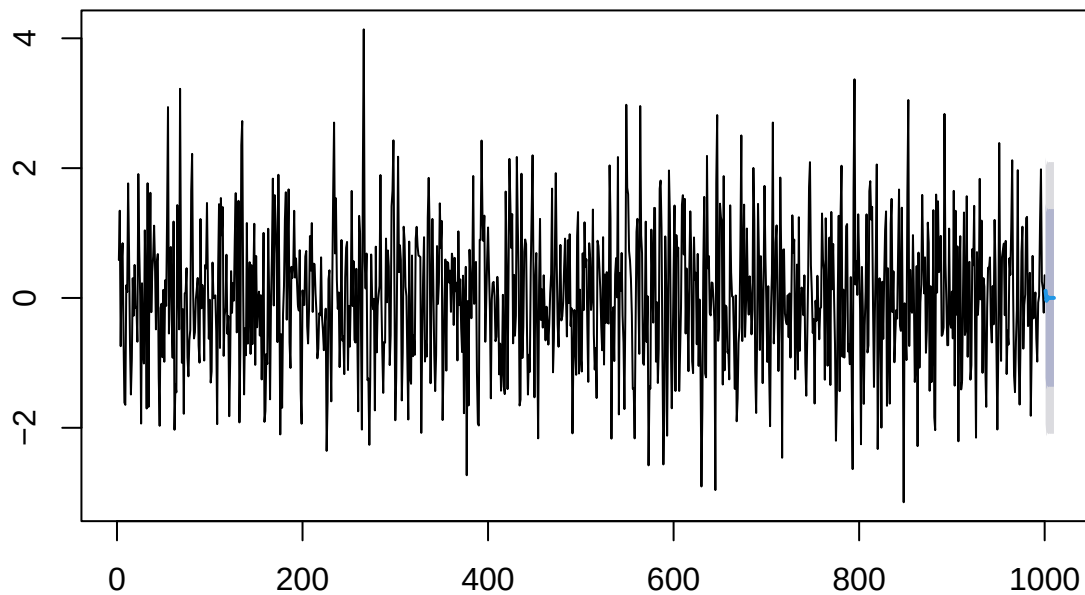
```
h <- log(98)
Box.test(z, lag=h, type="Ljung-Box")
```

```
##
## Box-Ljung test
##
## data: z
## X-squared = 17.42, df = 4, p-value = 0.001602
```

We see that p-value is still 0.0016, still some structure.

```
lhfc <- forecast(lh)
plot(lhfc, type="l", main="Forecast ARMA(1,1)")
```

Forecast ARMA(1,1)



Forecast is smoother, but still not based on stationary dynamics.

We then try ARMA(2, 2)

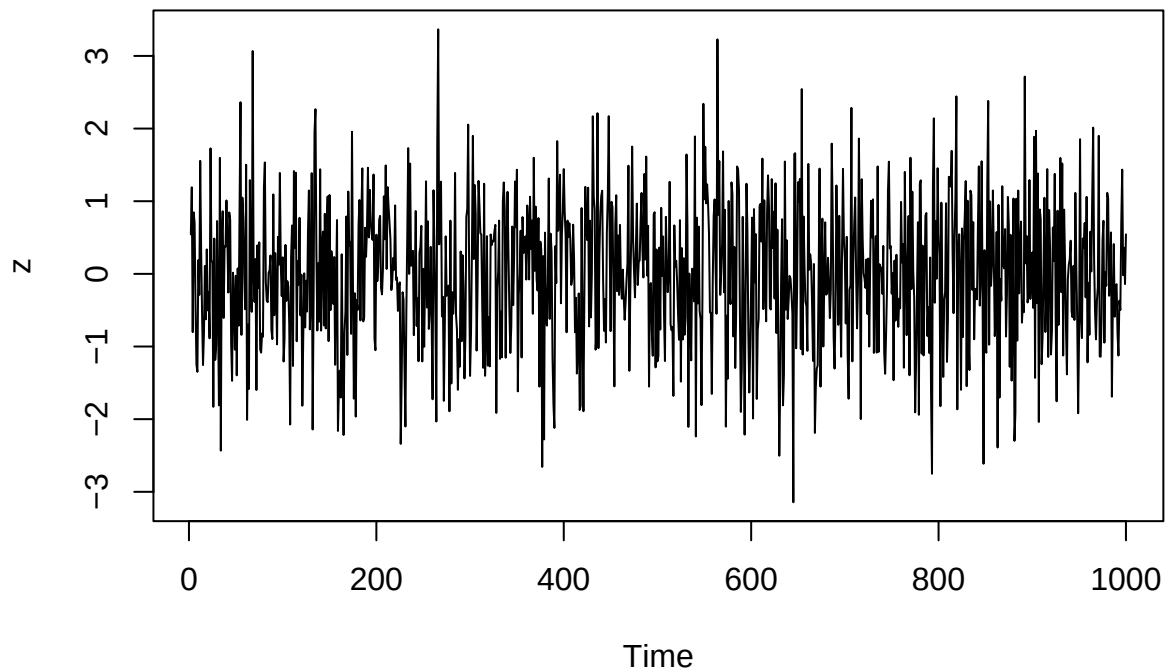
```
lh <- arima(y, order=c(2,0,2), method="ML")
lh

##
## Call:
## arima(x = y, order = c(2, 0, 2), method = "ML")
##
## Coefficients:
##          ar1      ar2      ma1      ma2  intercept
##          0.6567  0.0945 -0.5999 -0.4001         5e-04
## s.e.    0.1218  0.1075  0.1150  0.1149         6e-04
##
## sigma^2 estimated as 0.9885:  log likelihood = -1414.14,  aic = 2840.28
```

And residuals analysis:

```
z <- resid(lh)
plot(z, type="l", main="Residuals ARMA(2,2)")
```

Residuals ARMA(2,2)



Looks similar to previous residuals, sometimes slightly more irregular.

```
Box.test(z, lag=h, type="Ljung-Box")
```

```
##  
## Box-Ljung test  
##  
## data: z  
## X-squared = 0.11436, df = 4, p-value = 0.9984
```

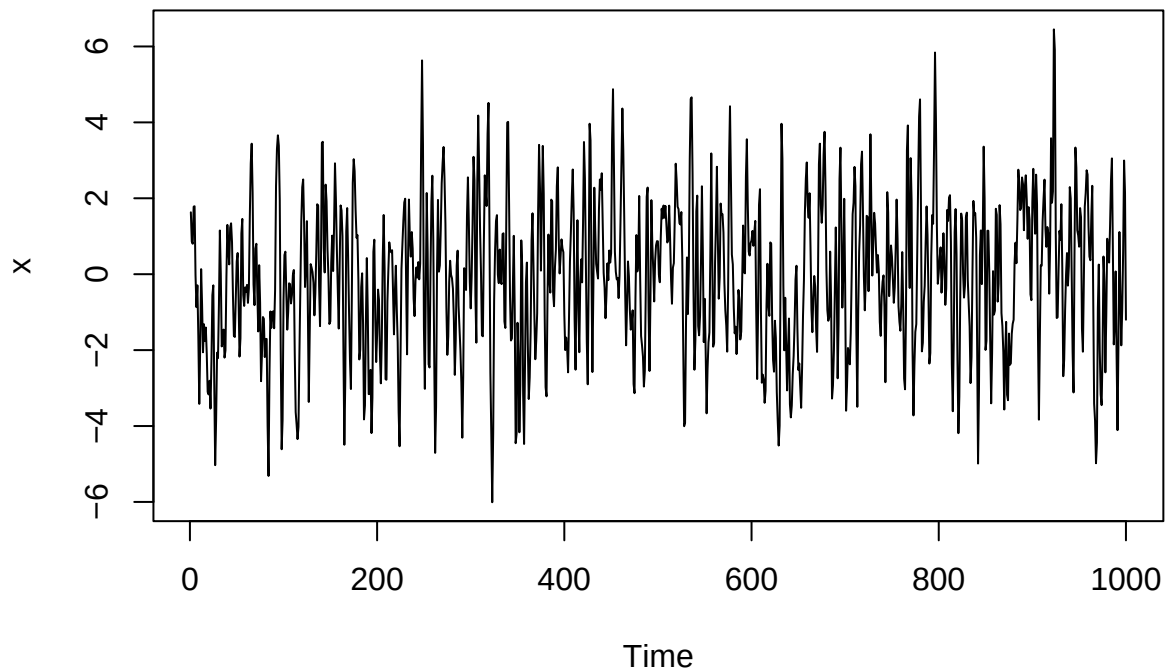
We obtain in this case a p-value of 0.9984 which is much larger than 0.05, so we do not reject the null hypothesis of no autocorrelation in the residuals. This suggests that the ARMA(2, 2) model provides a better fit to the data compared to the previous models, as it captures more of the dependence structure and leaves residuals that behave more like white noise.

B) Generate MA(2) and fit an MA(3) for example. Note that the third coefficient is statistically null. Note that is better to force mean=0 to avoid statistical noise. Try the following:

First we generate MA(2):

```
n <- 1000  
x <- arima.sim(list(order=c(0,0,2), ma=c(1.5,0.75)), n)  
plot(x, type="l", main="Simulated MA(2)")
```


Simulated MA(2)



Series shows short-term dependence and no trend (stationary).

Then we fit correct MA(2) model:

```
x.ma2 <- arima(x, order=c(0,0,2), include.mean=FALSE)
x.ma2

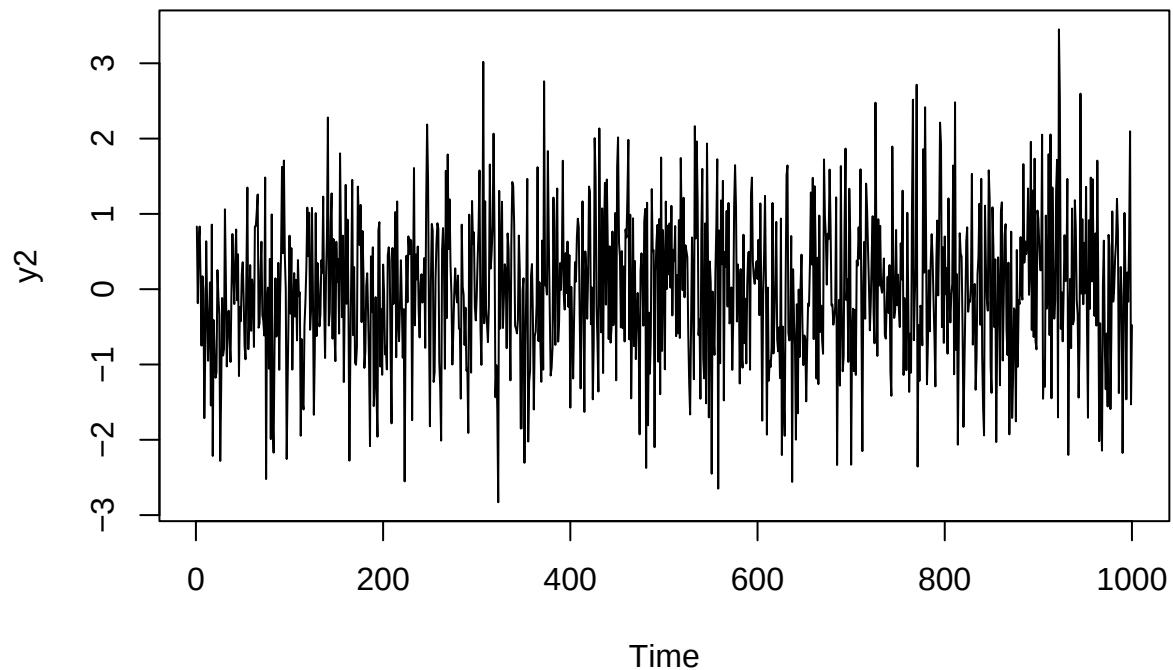
##
## Call:
## arima(x = x, order = c(0, 0, 2), include.mean = FALSE)
##
## Coefficients:
##          ma1      ma2
##       1.5089  0.7569
## s.e.  0.0203  0.0204
##
## sigma^2 estimated as 0.9772:  log likelihood = -1408.95,  aic = 2823.9
```

We obtain coefficients around 1.5 and 0.75, close to the true values.

And then we get the residuals:

```
y2 <- resid(x.ma2)
plot(y2, type="l", main="Residuals MA(2)")
```

Residuals MA(2)



```
Box.test(y2, lag=log(n), type="Ljung-Box")
```

```
##
## Box-Ljung test
##
## data: y2
## X-squared = 2.9297, df = 6, p-value = 0.8176
```

Residuals look random, with no visible structure. The p-value is high (0.8176), so we do not reject the null hypothesis of no autocorrelation in the residuals. This confirms that the fitted MA(2) model is adequate for the simulated data.

Now we fit an MA(3) model:

```
x.ma3 <- arima(x, order=c(0,0,3), include.mean=FALSE)
x.ma3
```

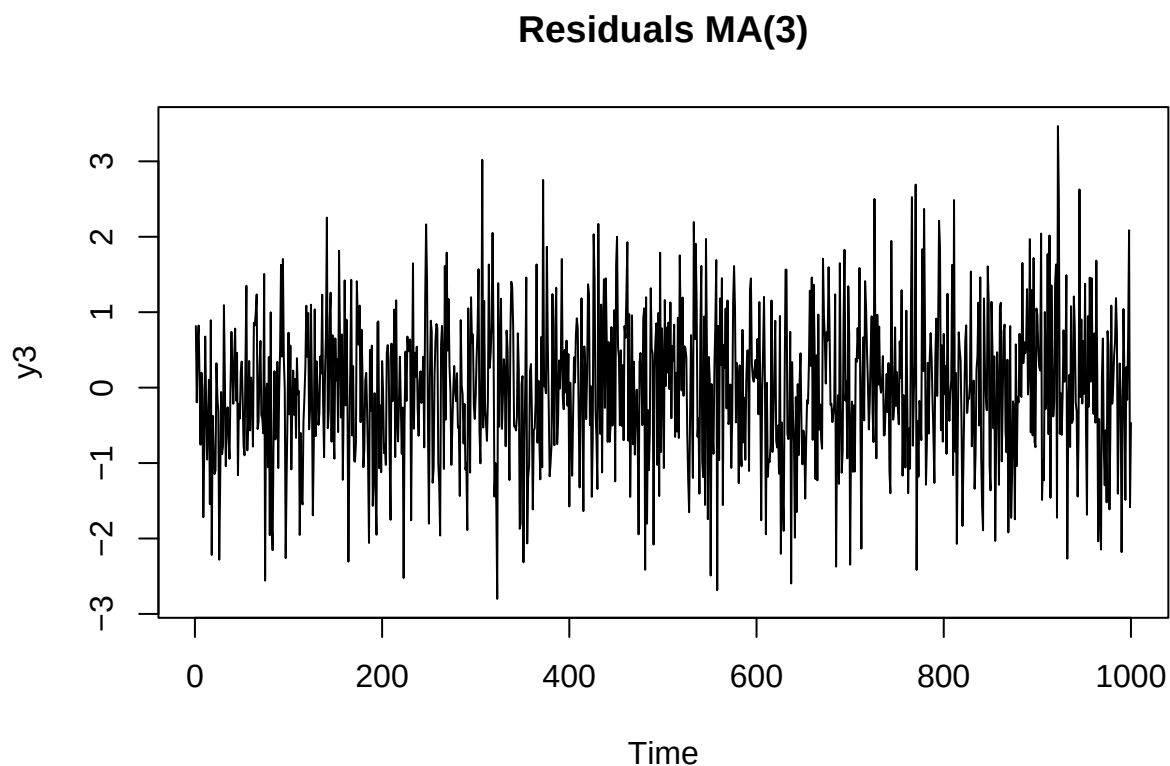
```
##
## Call:
## arima(x = x, order = c(0, 0, 3), include.mean = FALSE)
##
## Coefficients:
##          ma1      ma2      ma3
##       1.5323  0.8035  0.0310
## s.e.  0.0322  0.0534  0.0327
```

```
##  
## sigma^2 estimated as 0.9764: log likelihood = -1408.5, aic = 2825
```

The third coefficient is close to zero. This is expected, since the true model is MA(2), so the third MA term should not contribute significantly to the fit. There's a large standard error for the third coefficient, indicating that it is not statistically significant. This confirms that the MA(3) model does not provide a better fit than the MA(2) model, and the additional parameter is not necessary.

We compute the residuals of the MA(3) model:

```
y3 <- resid(x.ma3)  
plot(y3, type="l", main="Residuals MA(3)")
```



```
Box.test(y3, lag=log(n), type="Ljung-Box")
```

```
##  
## Box-Ljung test  
##  
## data: y3  
## X-squared = 2.1168, df = 6, p-value = 0.9086
```

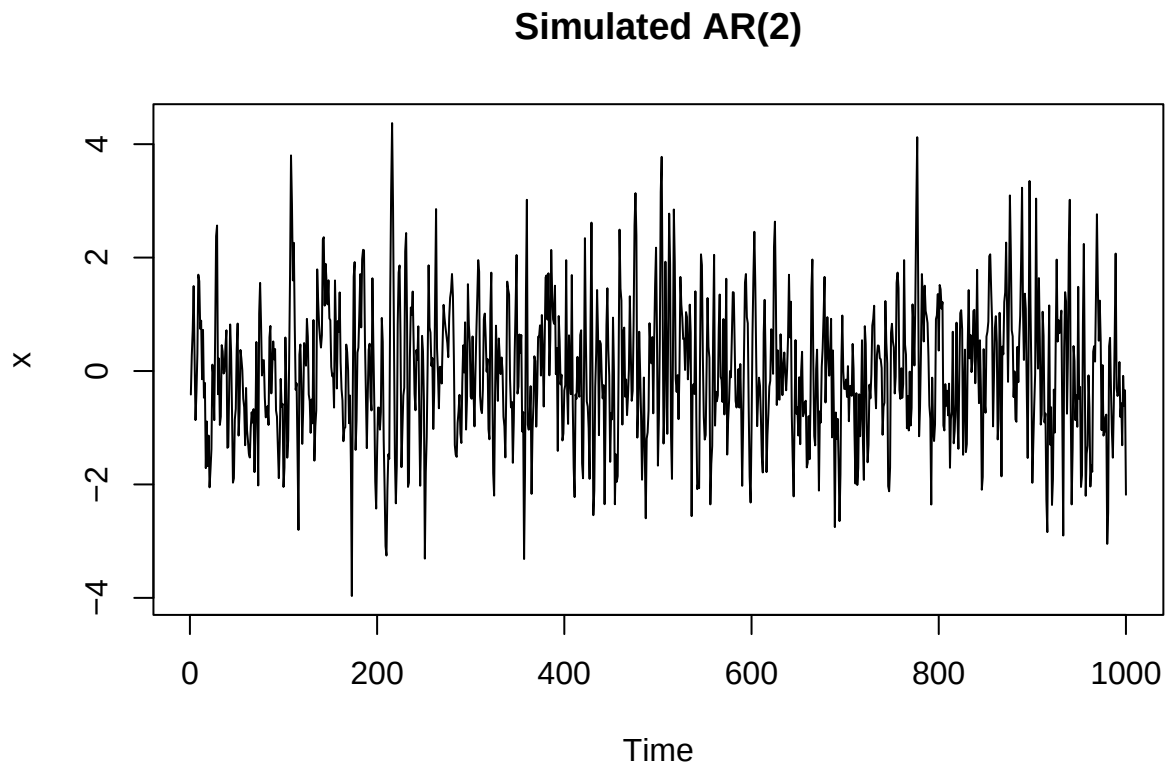
Residuals are very similar to MA(2) residuals, with no visible structure. The p-value is also high (0.8176), confirming that the MA(3) model also provides an adequate fit. However, since the third coefficient is not statistically significant, the simpler MA(2) model is preferred for parsimony.

Conclusion In this example, we simulated an MA(2) process and fitted both an MA(2) and an MA(3) model. The MA(2) model correctly identified the true parameters and produced residuals that behaved like white noise. The MA(3) model included an additional parameter that was not statistically significant, and its residuals were similar to those of the MA(2) model. Therefore, the MA(2) model is preferred for its simplicity and adequacy in capturing the underlying process.

C) Repeat the previous exercise with an AR model

We repeat the same steps with an AR(2) model instead.

```
n <- 1000
x <- arima.sim(list(order=c(2,0,0), ar=c(0.7, -0.3)), n)
plot(x, type="l", main="Simulated AR(2)")
```



We see that the series shows short-term dependence over time, and smooth fluctuations (typical AR behaviour).

Then we fit the correct AR(2) model:

```
x.ar2 <- arima(x, order=c(2,0,0), include.mean=FALSE)
x.ar2

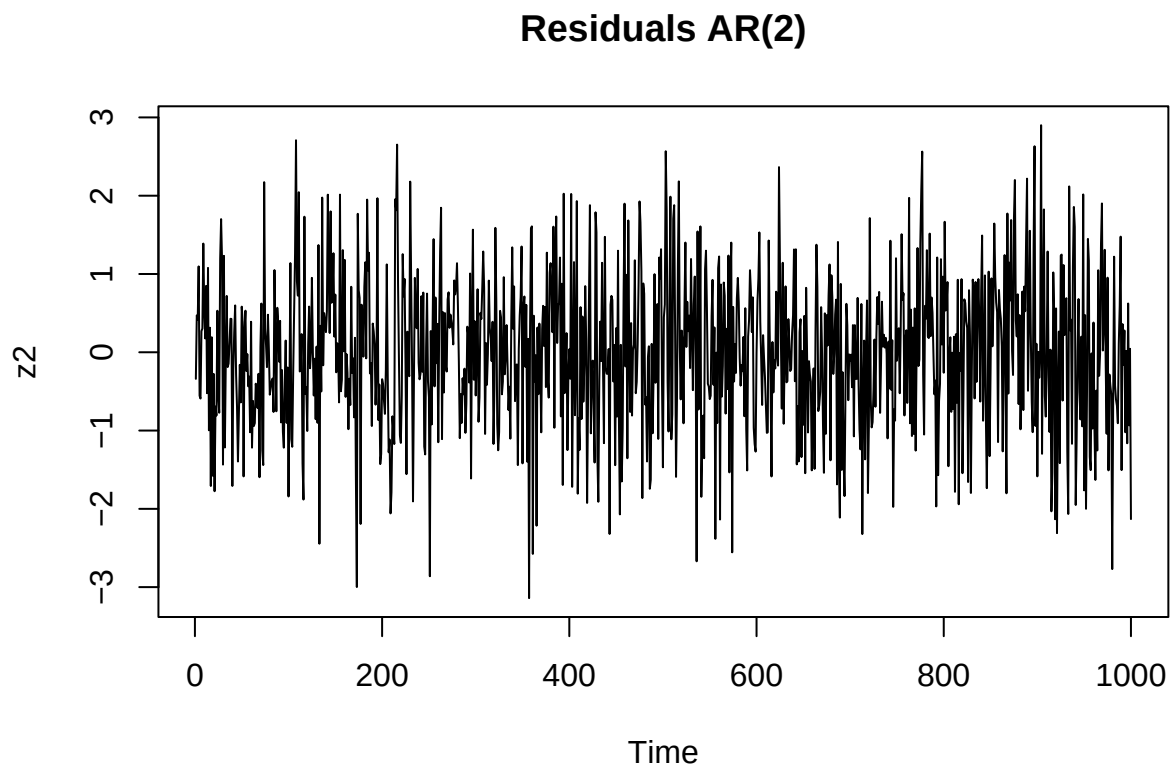
##
## Call:
## arima(x = x, order = c(2, 0, 0), include.mean = FALSE)
```

```
##
## Coefficients:
##      ar1      ar2
## 0.6668 -0.2878
## s.e. 0.0303 0.0303
##
## sigma^2 estimated as 0.982: log likelihood = -1410.1, aic = 2826.2
```

We obtain coefficients around 0.7 and -0.3, close to the true values.

And then we get the residuals:

```
z2 <- resid(x.ar2)
plot(z2, type="l", main="Residuals AR(2)")
```



```
Box.test(z2, lag=log(n), type="Ljung-Box")
```

```
##
## Box-Ljung test
##
## data: z2
## X-squared = 5.4886, df = 6, p-value = 0.4828
```

Residuals look random, with a p-value of (0.4828). This is larger than 0.05, so we do not reject the null hypothesis of no autocorrelation in the residuals. This confirms that the fitted AR(2) model is adequate for the simulated data.

Now we overfit with an AR(3) model:

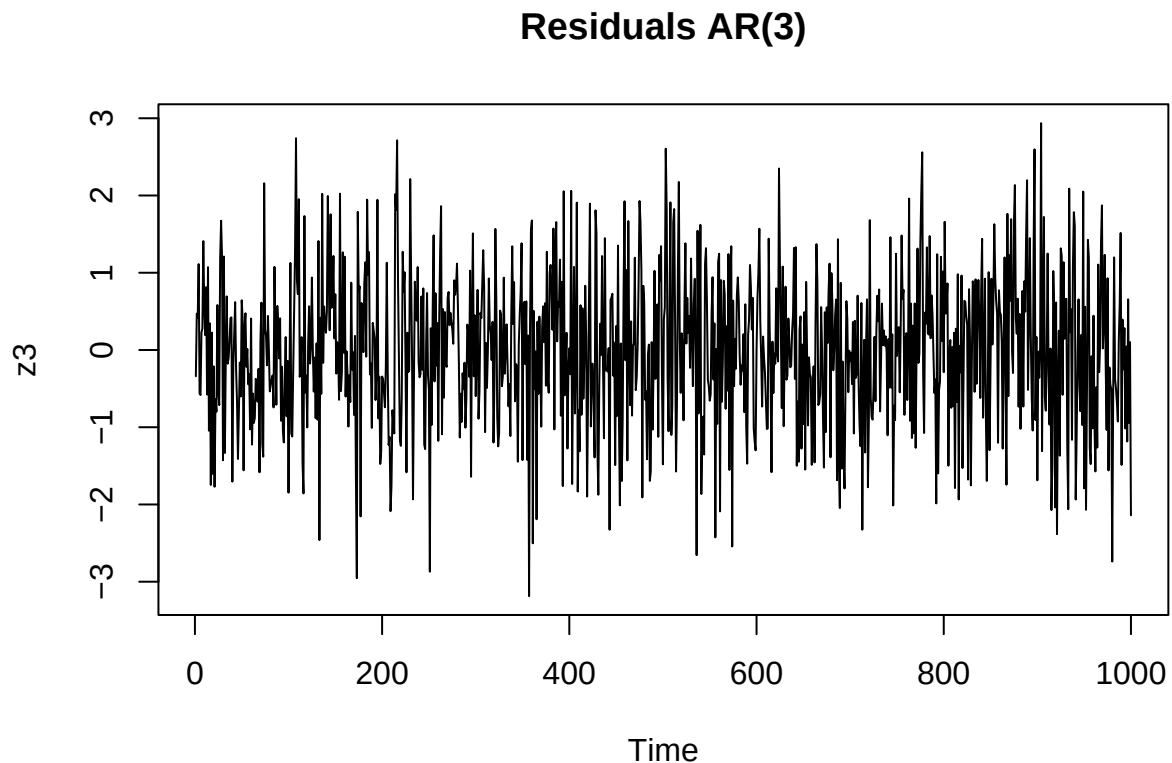
```
x.ar3 <- arima(x, order=c(3,0,0), include.mean=FALSE)
x.ar3
```

```
##
## Call:
## arima(x = x, order = c(3, 0, 0), include.mean = FALSE)
##
## Coefficients:
##          ar1      ar2      ar3
##      0.6785 -0.3147  0.0402
## s.e.  0.0317  0.0369  0.0316
##
## sigma^2 estimated as 0.9804:  log likelihood = -1409.29,  aic = 2826.58
```

The third coefficient is close to zero, with a large standard error, indicating that it is not statistically significant. This confirms that the AR(3) model does not provide a better fit than the AR(2) model, and the additional parameter is not necessary.

We compute the residuals of the AR(3) model:

```
z3 <- resid(x.ar3)
plot(z3, type="l", main="Residuals AR(3)")
```



```
Box.test(z3, lag=log(n), type="Ljung-Box")
```

```
##  
## Box-Ljung test  
##  
## data: z3  
## X-squared = 4.0538, df = 6, p-value = 0.6694
```

Residuals are very similar to AR(2) residuals, with no visible structure. The p-value is also high (0.6694), confirming that the AR(3) model also provides an adequate fit. However, since the third coefficient is not statistically significant, the simpler AR(2) model is preferred for parsimony.

Conclusion

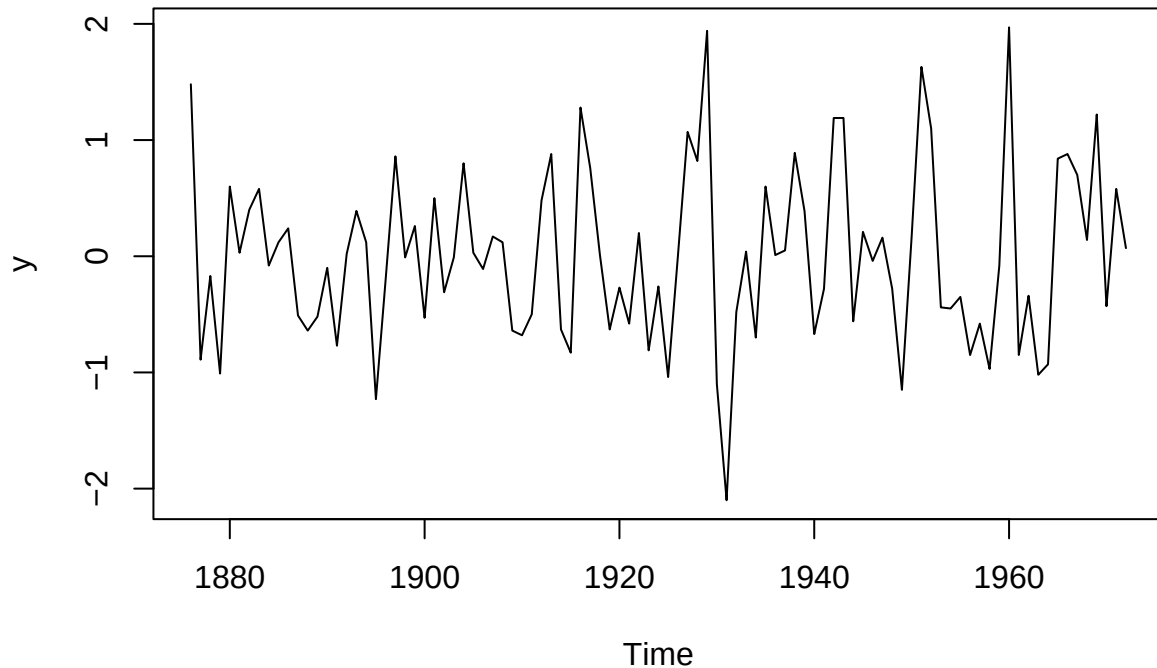
In this example, we simulated an AR(2) process and fitted both an AR(2) and an AR(3) model. The AR(2) model correctly identified the true parameters and produced residuals that behaved like white noise. The AR(3) model included an additional parameter that was not statistically significant, and its residuals were similar to those of the AR(2) model. Therefore, the AR(2) model is preferred for its simplicity and adequacy in capturing the underlying process.

D) Sometimes it may be difficult to distinguish more complex ARMA models like an ARMA(2, 1) and an ARMA(1, 2). Thus the fitting is frequently not that good. Fit an ARIMA(a, 1, b) model to Lake Huron data.

We first prepare the data:

```
x <- LakeHuron  
y <- diff(x)  
plot(y, type="l", main="Differenced Lake Huron")
```

Differenced Lake Huron



We removed the trend by taking first differences, so now we should have a more stationary series. We can now fit an ARIMA(a, 1, b) model to the original data, which is equivalent to fitting an ARMA(a, b) model to the differenced series y.

Then we fit an ARIMA(a, 1, b) model:

```
a <- 2
b <- 2

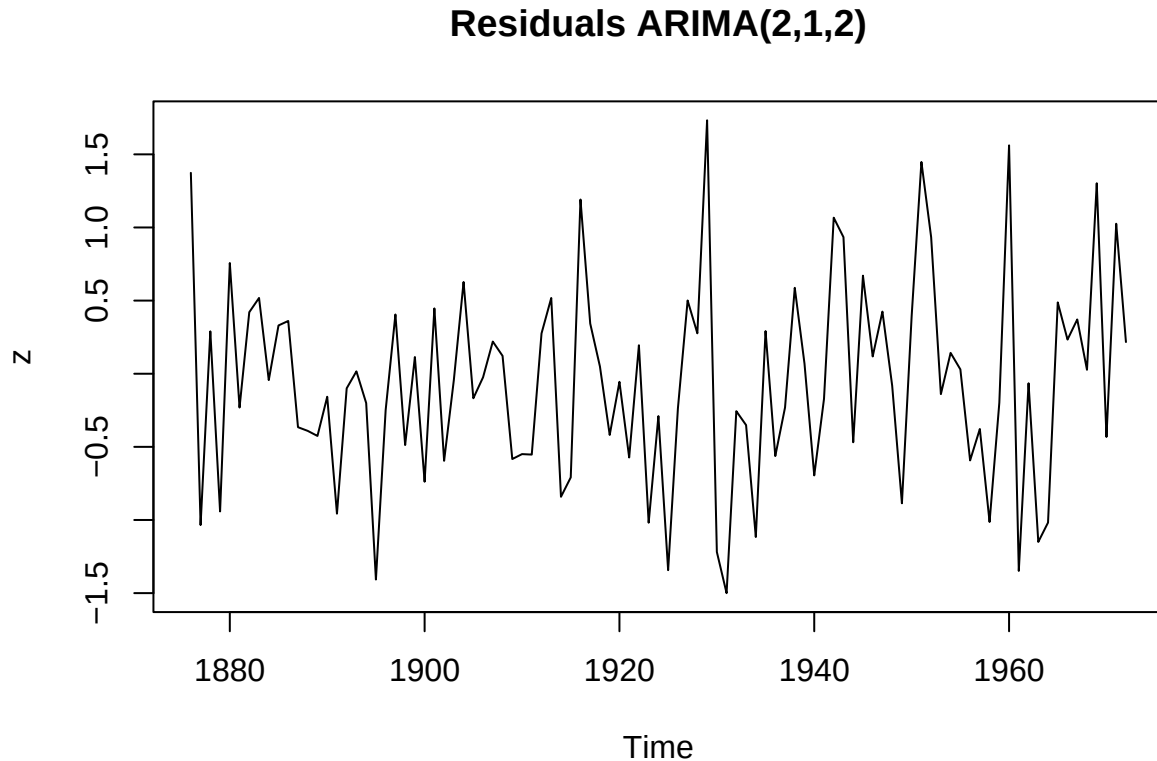
lh <- arima(y, order=c(a,0,b), method="ML", include.mean=FALSE)
lh

##
## Call:
## arima(x = y, order = c(a, 0, b), include.mean = FALSE, method = "ML")
##
## Coefficients:
##          ar1      ar2      ma1      ma2
##          0.8212 -0.1673 -0.7459 -0.1620
## s.e.      0.3175   0.2757   0.3184   0.3086
##
## sigma^2 estimated as 0.4799:  log likelihood = -102.4,  aic = 214.8
```

Model is relatively complex, with 4 parameters to estimate. The AR and MA coefficients are estimated, but some of them are not statistically significant (e.g. the second AR term). This suggests that a simpler model might be sufficient.

We compute the residuals:

```
z <- resid(lh)
plot(z, type="l", main="Residuals ARIMA(2,1,2)")
```



Residuals look more random than previous models, with no clear structure.

```
h <- 5
Box.test(z, lag=h, type="Ljung-Box")
```

```
##
## Box-Ljung test
##
## data: z
## X-squared = 0.24795, df = 5, p-value = 0.9985
```

The p-value is 0.9985, which is much larger than 0.05, so we do not reject the null hypothesis of no autocorrelation in the residuals. This suggests that the ARIMA(2, 1, 2) model provides a good fit to the data, as it captures the underlying structure and leaves residuals that behave like white noise.

We then change the lag h:

```
h <- log(98)
Box.test(z, lag=h, type="Ljung-Box")
```

```
##
## Box-Ljung test
##
## data: z
## X-squared = 0.22113, df = 4, p-value = 0.9943
```

And obtain approximately the same result, with a p-value of 0.9943, confirming that the residuals are not autocorrelated and the model is adequate.

Finally we reduce the model order trying smaller models

```
model1 <- arima(y, order=c(1,0,1), include.mean=FALSE)
model2 <- arima(y, order=c(1,0,0), include.mean=FALSE)
model3 <- arima(y, order=c(0,0,1), include.mean=FALSE)
```

And check the residuals of these simpler models to see whether they are still adequate or not.

```
Box.test(resid(model1), lag=5, type="Ljung-Box")
```

```
##
## Box-Ljung test
##
## data: resid(model1)
## X-squared = 7.0407, df = 5, p-value = 0.2176
```

```
Box.test(resid(model2), lag=log(98), type="Ljung-Box")
```

```
##
## Box-Ljung test
##
## data: resid(model2)
## X-squared = 6.8856, df = 4, p-value = 0.1421
```

```
Box.test(resid(model3), lag=log(98), type="Ljung-Box")
```

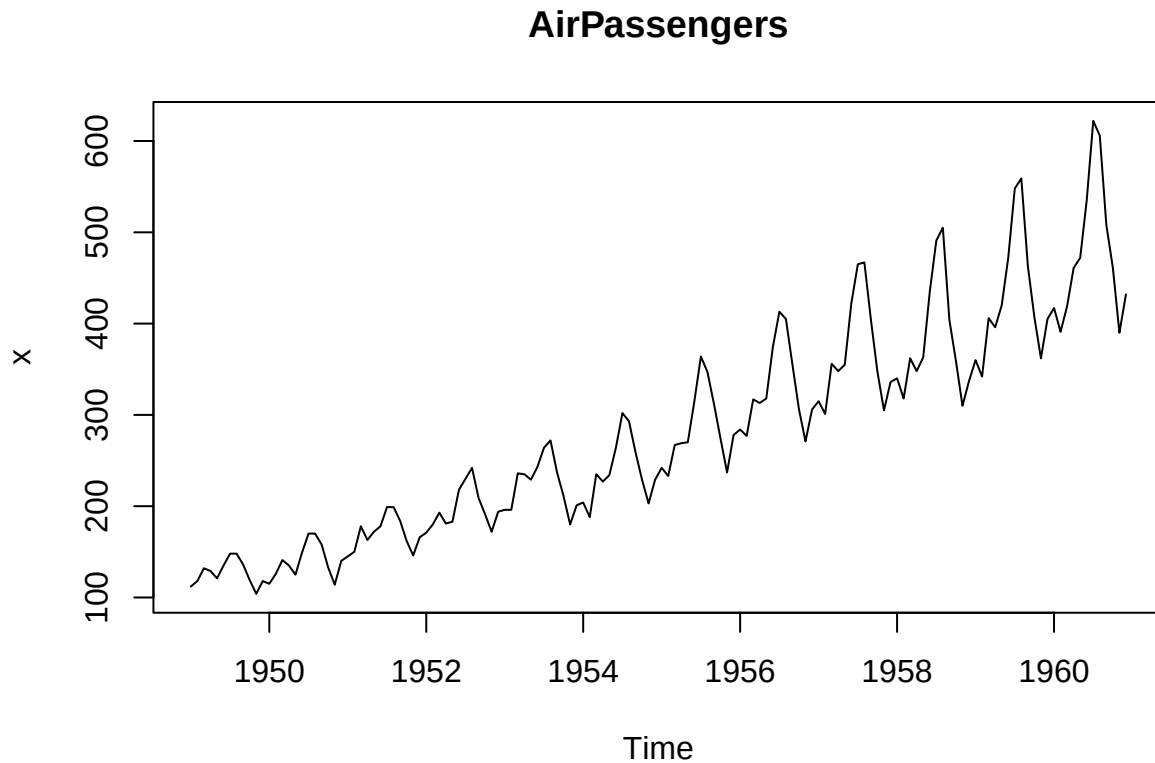
```
##
## Box-Ljung test
##
## data: resid(model3)
## X-squared = 5.2869, df = 4, p-value = 0.2591
```

As we decrease (a, b) the residuals look less random and the p-value decreases. This shows that simpler models fail to capture structure and more complex models (2, 2) perform better.

E) Fit ARIMA models to other datasets

We will use the dataset of practical 1, AirPassengers

```
library(datasets)
x <- AirPassengers
plot(x, type="l", main="AirPassengers")
```



We see a clear increasing trend, clear seasonality and no stationarity.

We can try an automatic ARIMA fitting first:

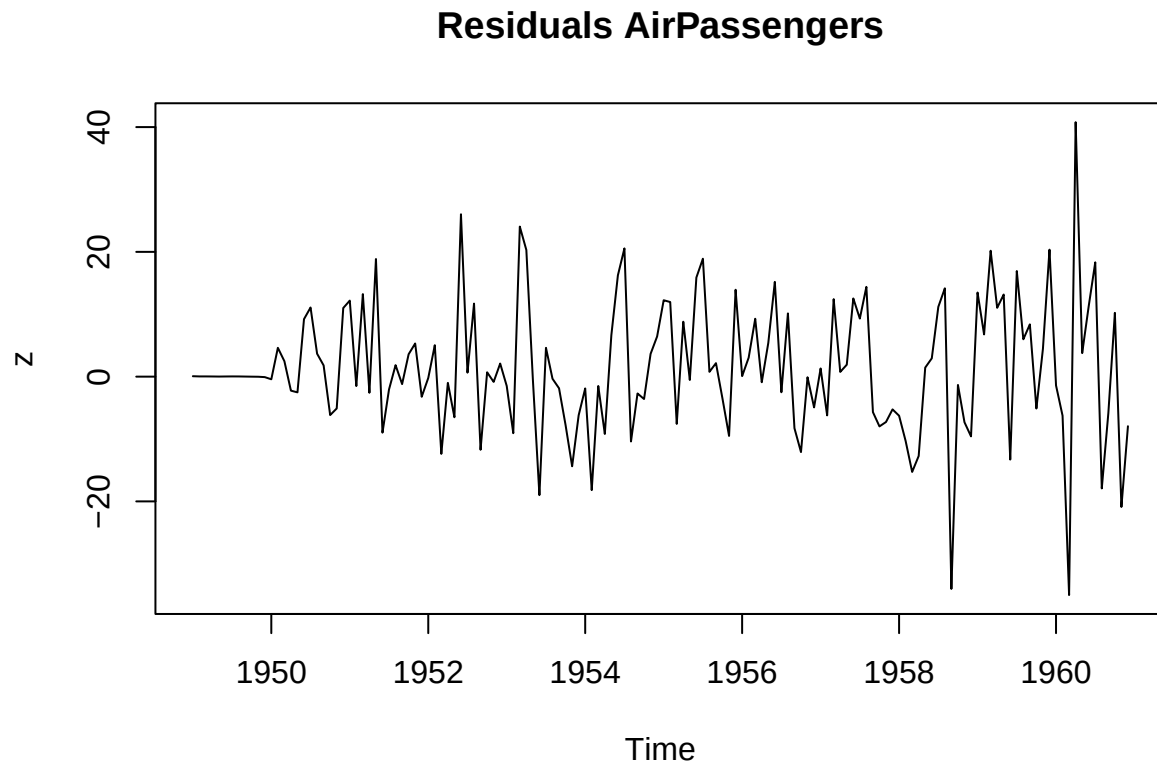
```
fit <- auto.arima(x)
fit

## Series: x
## ARIMA(2,1,1)(0,1,0)[12]
##
## Coefficients:
##      ar1      ar2      ma1
##      0.5960  0.2143 -0.9819
## s.e.  0.0888  0.0880  0.0292
##
## sigma^2 = 132.3: log likelihood = -504.92
## AIC=1017.85  AICc=1018.17  BIC=1029.35
```

Model is ARIMA(2,1,1)(0,1,0)[12] with non-zero mean. This is a complex model with many parameters, but it is necessary to capture the trend and seasonality in the data.

We then compute the residuals:

```
z <- resid(fit)
plot(z, type="l", main="Residuals AirPassengers")
```



```
Box.test(z, lag=log(length(x)), type="Ljung-Box")
```

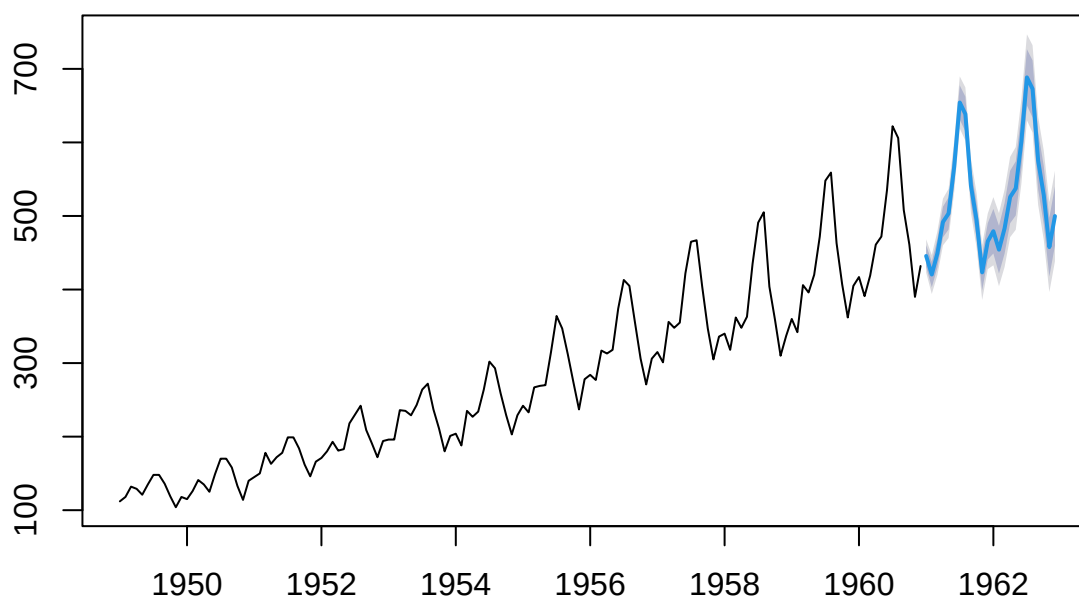
```
##
## Box-Ljung test
##
## data: z
## X-squared = 2.4796, df = 4, p-value = 0.6483
```

Residuals look random and p-value is 0.6483, higher than 0.05, so we do not reject the null hypothesis of no autocorrelation in the residuals. This confirms that the fitted ARIMA(2,1,1)(0,1,0)[12] model is adequate for the AirPassengers data.

Then we forecast:

```
fc <- forecast(fit)
plot(fc, main="Forecast AirPassengers")
```

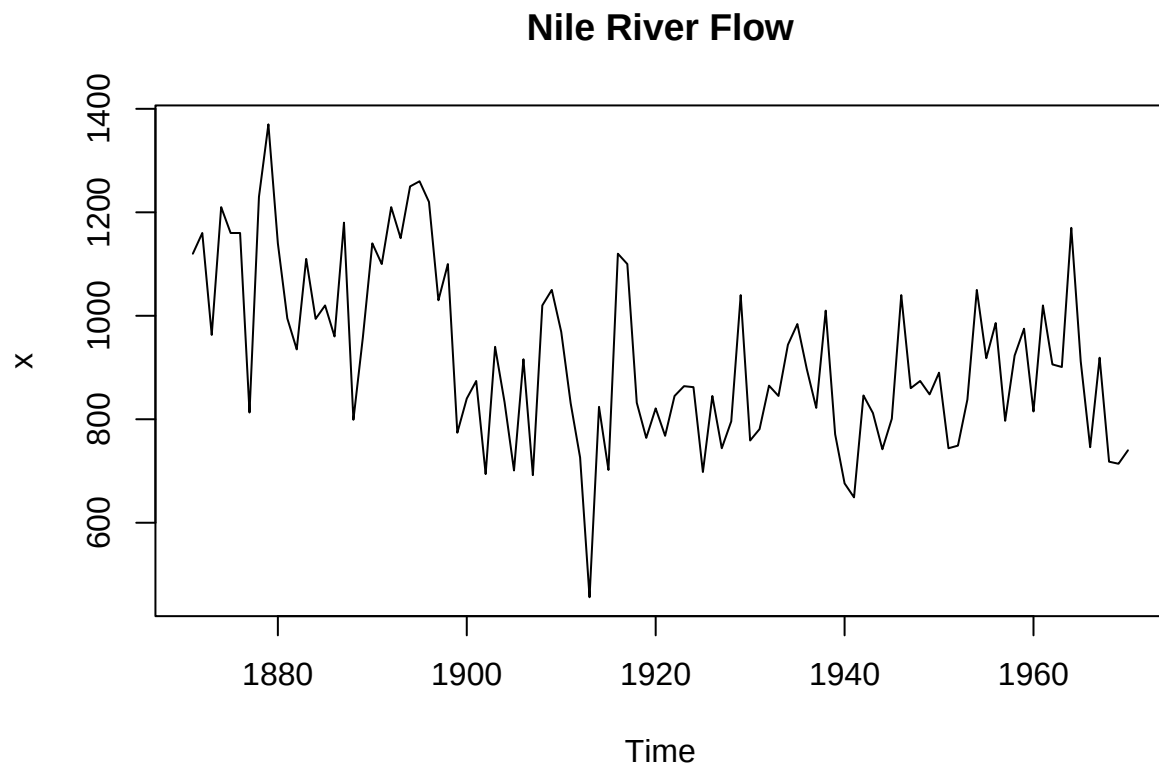
Forecast AirPassengers



Forecast shows an increasing trend and seasonal pattern, consistent with the structure captured by the model. The prediction intervals widen as time goes on, reflecting increasing uncertainty in the forecasts.

We can check another dataset not used before, like the Nile River Flow.

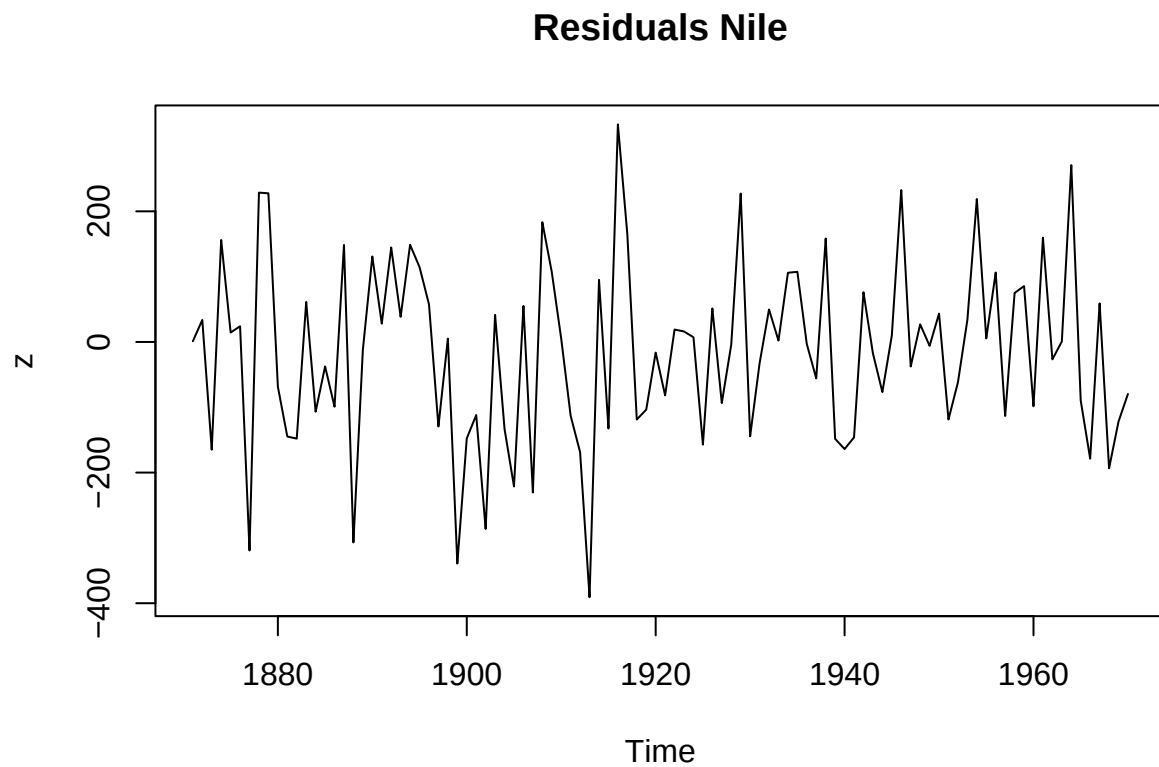
```
x <- Nile
plot(x, type="l", main="Nile River Flow")
```



```
fit <- auto.arima(x)
fit
```

```
## Series: x
## ARIMA(1,1,1)
##
## Coefficients:
##      ar1      ma1
##      0.2544 -0.8741
## s.e.  0.1194  0.0605
##
## sigma^2 = 20177: log likelihood = -630.63
## AIC=1267.25  AICc=1267.51  BIC=1275.04
```

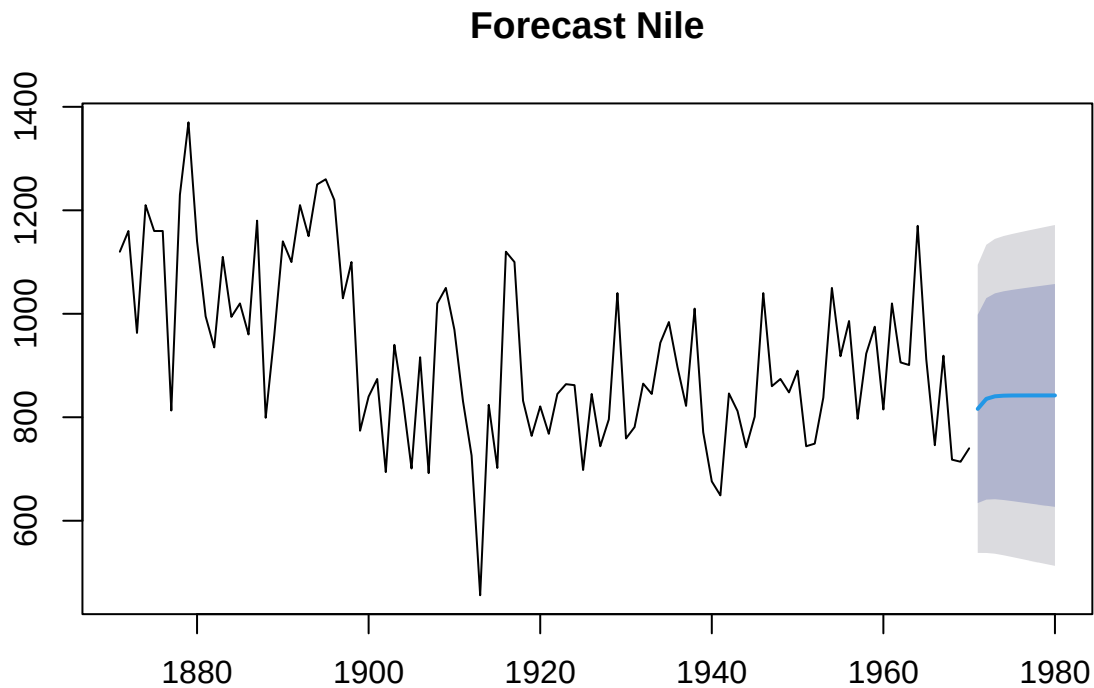
```
z <- resid(fit)
plot(z, type="l", main="Residuals Nile")
```



```
Box.test(z, lag=log(length(x)), type="Ljung-Box")
```

```
##  
## Box-Ljung test  
##  
## data: z  
## X-squared = 1.1002, df = 4, p-value = 0.8942
```

```
fc <- forecast(fit)  
plot(fc, main="Forecast Nile")
```



For this second example a relatively simple ARIMA model is sufficient for the Nile data. The residuals behave like white noise, indicating that the model capture the structure of the series.

Practical 4b: Dickey-Fuller test and SARIMA models

A) The Dickey-Fuller test:

```
n<-200
x<-rnorm(n)
x
```

```
## [1] 0.32696686 1.10772290 0.76616288 1.05367298 -1.35594280 -0.16930139
## [7] -0.06970099 0.72019565 -0.16778188 -0.20327892 1.67812825 1.09093513
## [13] -1.75644463 -0.38461079 -0.99215819 2.97158503 -0.49433453 1.14803978
## [19] 0.09627125 0.10883021 0.49523695 -0.14264350 0.83293700 0.55982377
## [25] -1.68509595 -0.55561231 -0.52335312 -0.50610433 1.06528489 1.48702703
## [31] -0.92180095 0.54143547 -1.16976793 -0.55708038 0.29846554 -0.18892279
## [37] -0.68045020 -1.25744854 -0.31176654 0.05249805 0.56223381 -0.87581682
## [43] 0.58675596 -1.16383957 -0.54611202 0.45037717 -0.69251547 -0.77959611
## [49] -0.04733266 0.03543457 0.93448196 -0.23338602 -1.70078808 -0.65583810
## [55] -0.63047047 -0.93232759 0.32190370 2.50176510 0.79416747 -0.15483956
## [61] 0.50366718 1.44276824 -0.30926718 0.66448047 0.56189912 -0.12263483
## [67] 0.05654601 0.65189100 -0.02623864 -0.33950900 -0.59398998 0.29096208
```



```
## [73] -0.05502200 -0.37173828 0.17658847 -0.77306692 -0.80241961 1.37356613
## [79] -0.88135606 0.31578806 -0.96838658 -0.23060965 0.39036228 -1.00953678
## [85] 0.59503867 0.21891523 -0.31660629 0.24546654 0.07504484 -0.83965003
## [91] -1.39706392 0.10613177 -1.00472117 -0.03521999 -1.64807617 -0.92903249
## [97] 0.27211081 1.04532279 0.16927855 -0.99443498 -0.41533740 1.09363613
## [103] 0.51868426 0.78704034 0.99670095 0.37746798 1.10938000 -0.97321396
## [109] 0.29964526 -0.33948232 0.20173890 1.32539797 0.50379348 -0.62963669
## [115] -0.35015411 -0.95133863 -0.08981425 -0.49959690 0.79157269 -0.49272760
## [121] 0.71031471 0.72073013 -0.43533022 1.42649174 0.02692431 -0.65281842
## [127] 0.07439935 -0.99096252 0.65099328 -0.89516799 1.29299294 -2.07420659
## [133] -1.11246012 -0.33834589 -0.70069752 1.34694517 -0.06042597 0.35480442
## [139] 0.70736956 0.15287795 0.96101004 0.43971623 0.69821380 -1.48600746
## [145] -1.12632173 -2.22640749 -0.25327286 1.43175650 -0.97840283 0.31506322
## [151] 0.44095616 0.23852640 -0.28422261 -0.61814404 -0.63676796 0.01745325
## [157] 1.29963841 -0.79350749 -0.12253439 0.09926816 0.79141349 -0.23132812
## [163] 0.63771731 -1.49673281 0.71839966 0.09637101 -1.09564527 -2.33035864
## [169] -0.36533663 0.66886073 0.31905530 -0.36416639 0.05006536 0.15599060
## [175] -0.75241053 0.05455508 0.11226855 -0.72283146 0.19819556 0.31056031
## [181] 0.52632360 0.71104652 0.41031061 0.30139893 -0.09543010 0.44876031
## [187] -1.26924504 0.65427019 -0.53490937 2.33752882 -0.59633806 -2.88762983
## [193] 1.37208530 -0.59865238 0.59295092 0.22574207 1.09631206 -0.90326602
## [199] -1.18906159 1.06496900
```

```
library(tseries)
adf.test(x)
```

```
##
## Augmented Dickey-Fuller Test
##
## data: x
## Dickey-Fuller = -6.1464, Lag order = 5, p-value = 0.01
## alternative hypothesis: stationary
```

```
pp.test(x)
```

```
##
## Phillips-Perron Unit Root Test
##
## data: x
## Dickey-Fuller Z(alpha) = -197.23, Truncation lag parameter = 4, p-value
## = 0.01
## alternative hypothesis: stationary
```

Observations:

As p-value is very small (0.01), the hypothesis of random walk is rejected.

We will now create a random walk with a cumulative sum:

```
y<-cumsum(x)
y
```

```
## [1] 0.326966861 1.434689759 2.200852636 3.254525618 1.898582819
```

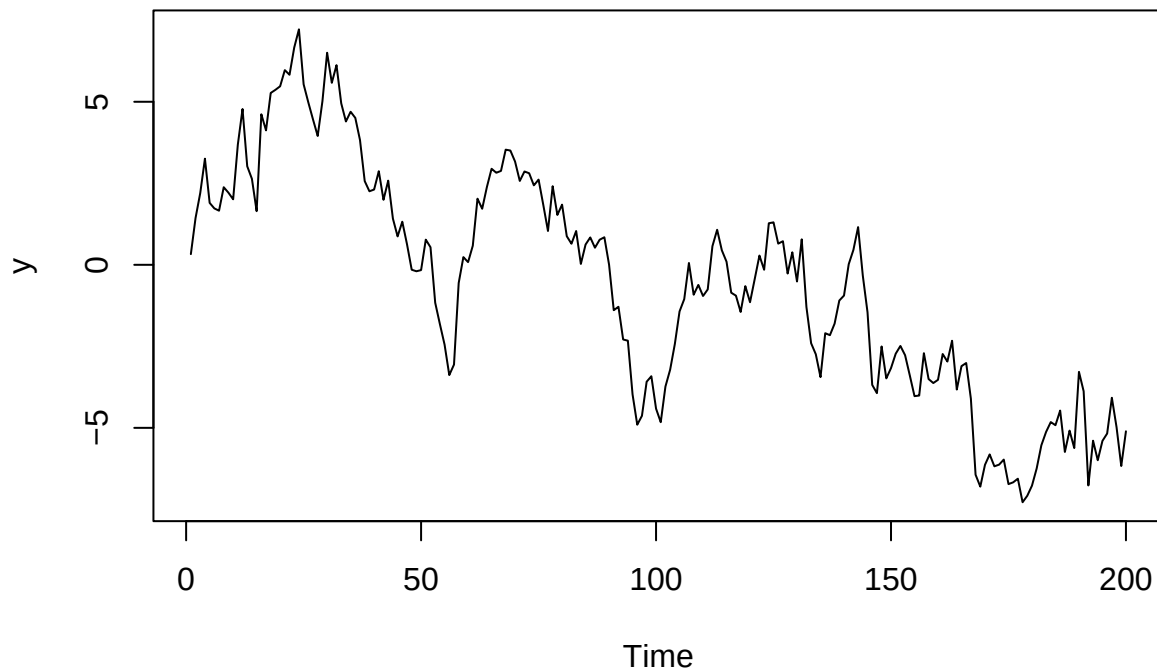
```

## [6] 1.729281433 1.659580445 2.379776094 2.211994217 2.008715301
## [11] 3.686843551 4.777778679 3.021334049 2.636723256 1.644565069
## [16] 4.616150102 4.121815571 5.269855354 5.366126603 5.474956814
## [21] 5.970193763 5.827550265 6.660487261 7.220311035 5.535215089
## [26] 4.979602776 4.456249653 3.950145327 5.015430212 6.502457246
## [31] 5.580656297 6.122091763 4.952323837 4.395243460 4.693709004
## [36] 4.504786214 3.824336011 2.566887470 2.255120933 2.307618979
## [41] 2.869852791 1.994035968 2.580791931 1.416952360 0.870840338
## [46] 1.321217507 0.628702033 -0.150894076 -0.198226736 -0.162792170
## [51] 0.771689789 0.538303768 -1.162484315 -1.818322413 -2.448792882
## [56] -3.381120471 -3.059216769 -0.557451673 0.236715799 0.081876239
## [61] 0.585543423 2.028311664 1.719044487 2.383524957 2.945424074
## [66] 2.822789248 2.879335254 3.531226257 3.504987614 3.165478618
## [71] 2.571488638 2.862450714 2.807428717 2.435690439 2.612278909
## [76] 1.839211987 1.036792379 2.410358511 1.529002449 1.844790509
## [81] 0.876403931 0.645794284 1.036156566 0.026619782 0.621658453
## [86] 0.840573682 0.523967391 0.769433934 0.844478769 0.004828735
## [91] -1.392235185 -1.286103416 -2.290824587 -2.326044576 -3.974120741
## [96] -4.903153231 -4.631042417 -3.585719622 -3.416441073 -4.410876056
## [101] -4.826213454 -3.732577327 -3.213893070 -2.426852731 -1.430151777
## [106] -1.052683800 0.056696198 -0.916517758 -0.616872502 -0.956354824
## [111] -0.754615928 0.570782042 1.074575525 0.444938831 0.094784718
## [116] -0.856553909 -0.946368160 -1.445965058 -0.654392363 -1.147119968
## [121] -0.436805257 0.283924877 -0.151405339 1.275086404 1.302010710
## [126] 0.649192288 0.723591639 -0.267370880 0.383622400 -0.511545588
## [131] 0.781447347 -1.292759239 -2.405219361 -2.743565254 -3.444262774
## [136] -2.097317602 -2.157743574 -1.802939159 -1.095569601 -0.942691650
## [141] 0.018318388 0.458034615 1.156248419 -0.329759046 -1.456080780
## [146] -3.682488266 -3.935761129 -2.504004633 -3.482407465 -3.167344243
## [151] -2.726388079 -2.487861680 -2.772084294 -3.390228337 -4.026996297
## [156] -4.009543050 -2.709904639 -3.503412124 -3.625946511 -3.526678355
## [161] -2.735264865 -2.966592982 -2.328875674 -3.825608482 -3.107208827
## [166] -3.010837819 -4.106483094 -6.436841730 -6.802178365 -6.133317633
## [171] -5.814262337 -6.178428727 -6.128363370 -5.972372768 -6.724783294
## [176] -6.670228216 -6.557959669 -7.280791129 -7.082595565 -6.772035251
## [181] -6.245711650 -5.534665128 -5.124354514 -4.822955587 -4.918385684
## [186] -4.469625378 -5.738870421 -5.084600227 -5.619509602 -3.281980779
## [191] -3.878318838 -6.765948667 -5.393863367 -5.992515748 -5.399564830
## [196] -5.173822761 -4.077510698 -4.980776722 -6.169838314 -5.104869313

```

```
ts.plot(y, type="l", main="Random walk",ylab="y",xlab="Time")
```

Random walk



```
adf.test(y)
```

```
##
## Augmented Dickey-Fuller Test
##
## data: y
## Dickey-Fuller = -3.5452, Lag order = 5, p-value = 0.03984
## alternative hypothesis: stationary
```

```
pp.test(y)
```

```
##
## Phillips-Perron Unit Root Test
##
## data: y
## Dickey-Fuller Z(alpha) = -25.481, Truncation lag parameter = 4, p-value
## = 0.01984
## alternative hypothesis: stationary
```

As expected, in this case the p-value is large and so we cannot reject the random walk hypothesis. Non-stationarity (random walk) is confirmed.

We now try to take first differences to adjust an ARMA model:

1. Simulate and AR(1) model with $\phi=0.9$

2. Compare different values of ϕ , 0.9, 0.99,... and different values of n (e.g. $n=1000$ and $\phi=0.9$. Then, $\phi=0.99$. And then $n=10,000$).

First we create a helper function: it simulates an AR(1), plots it, runs ADF test, # and also tests first differences

```
check_ar1 <- function(phi, n) {

  # Simulate AR(1):  $X_t = \phi * X_{t-1} + e_t$ 
  x <- arima.sim(model = list(ar = phi), n = n)

  # Plot original series
  plot(
    x, type = "l",
    main = paste("AR(1) series with  $\phi$  =", phi, "and  $n$  =", n),
    xlab = "Time", ylab = "x"
  )

  # ADF test on original series
  adf_x <- adf.test(x)

  cat("\n-----\n")
  cat("AR(1) with  $\phi$  =", phi, "and  $n$  =", n, "\n")
  cat("-----\n")
  print(adf_x)

  # First differences
  dx <- diff(x)

  # Plot differenced series
  plot(
    dx, type = "l",
    main = paste("First differences of AR(1),  $\phi$  =", phi, ",  $n$  =", n),
    xlab = "Time", ylab = "diff(x)"
  )

  # ADF test on differenced series
  adf_dx <- adf.test(dx)

  cat("\nADF test on first differences:\n")
  print(adf_dx)
}
```

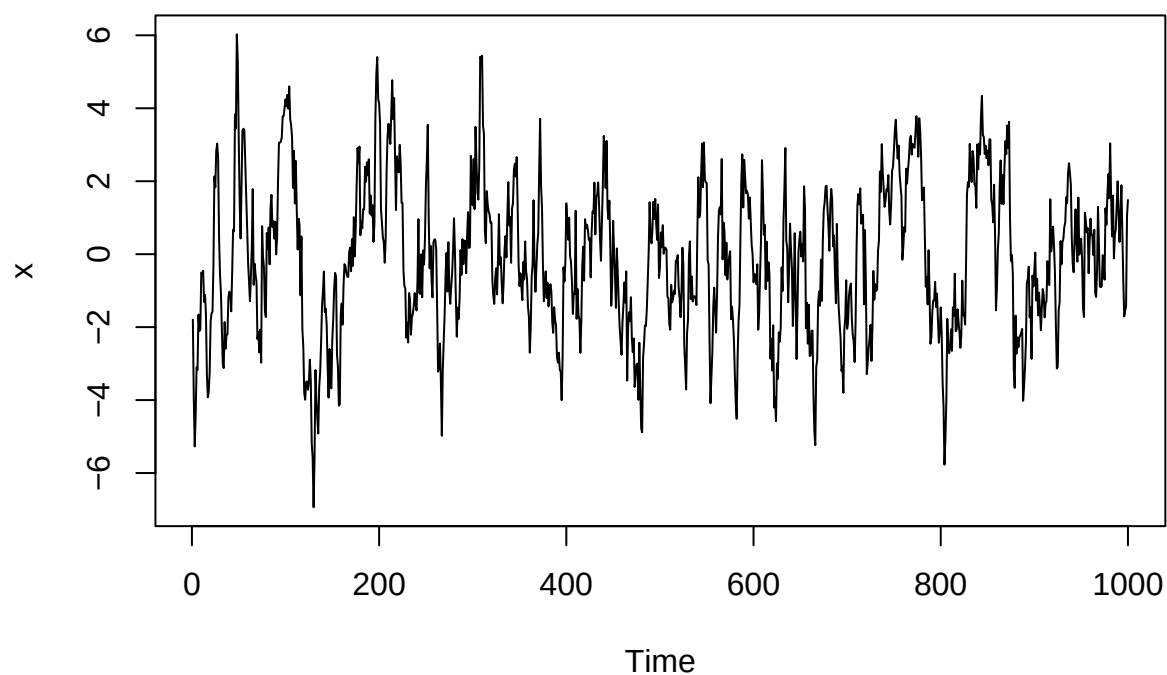
- **CASE 1:** $n = 1000$, $\phi = 0.9$

This is a stationary AR(1), because $\phi < 1$. So the ADF test should give a small p-value and so reject the random-walk hypothesis - so the series should be stationary. This is confirmed by the outputs below.

Also the plot of $\text{diff}(x)$ fluctuates around zero, indicating stationarity.

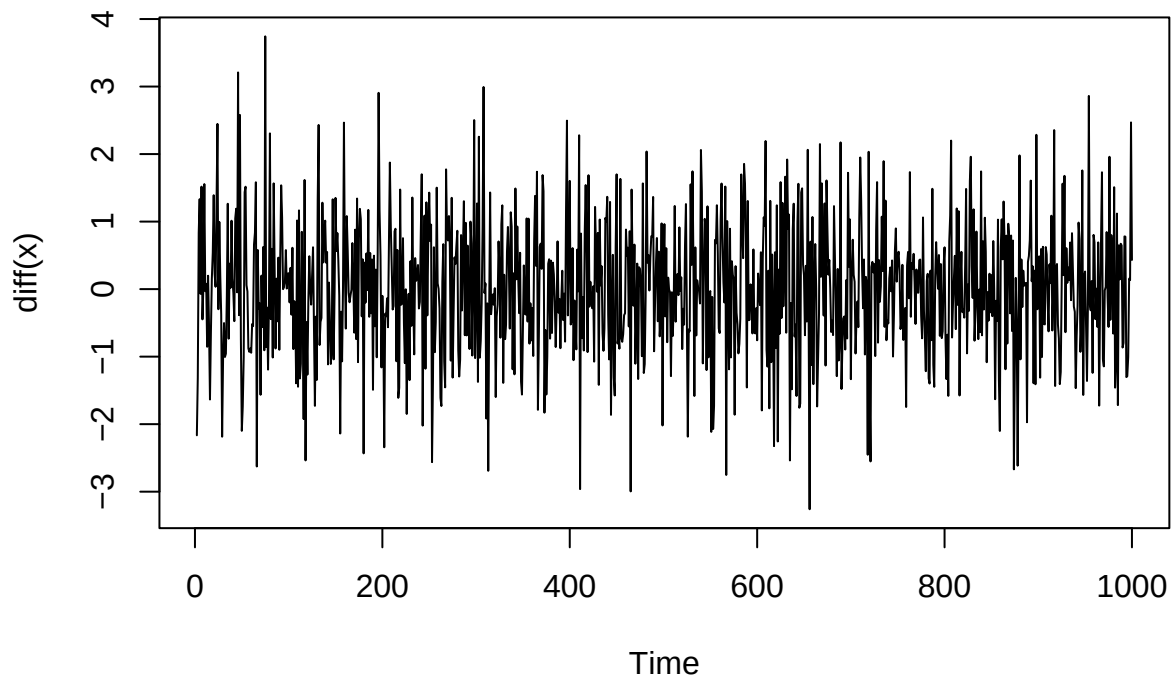
```
res_09_1000 <- check_ar1(phi = 0.9, n = 1000)
```

AR(1) series with $\phi = 0.9$ and $n = 1000$



```
##
## -----
## AR(1) with  $\phi = 0.9$  and  $n = 1000$ 
## -----
##
## Augmented Dickey-Fuller Test
##
## data: x
## Dickey-Fuller = -6.644, Lag order = 9, p-value = 0.01
## alternative hypothesis: stationary
```

First differences of AR(1), $\phi = 0.9$, $n = 1000$



```
##  
## ADF test on first differences:  
##  
## Augmented Dickey-Fuller Test  
##  
## data: dx  
## Dickey-Fuller = -11.816, Lag order = 9, p-value = 0.01  
## alternative hypothesis: stationary
```

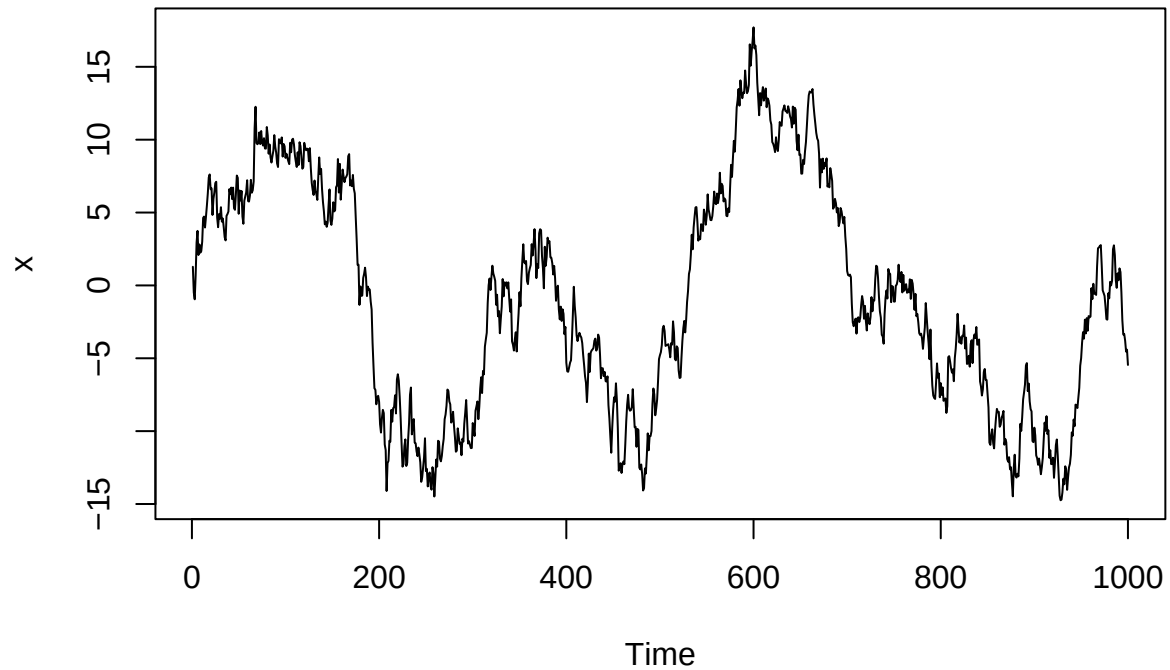
- **CASE 2:** $n = 1000$, $\phi = 0.99$ (increase ϕ)

This is still stationary in theory, because $\phi < 1$. The output below in fact still shows a small p-value but larger (< 0.5) so visually it starts to look more like a random walk - but the random-walk hypothesis is still rejected and the series is stationary.

The plot of $\text{diff}(x)$ still fluctuates around zero, indicating stationarity, however it has more evident fluctuations than the previous case.

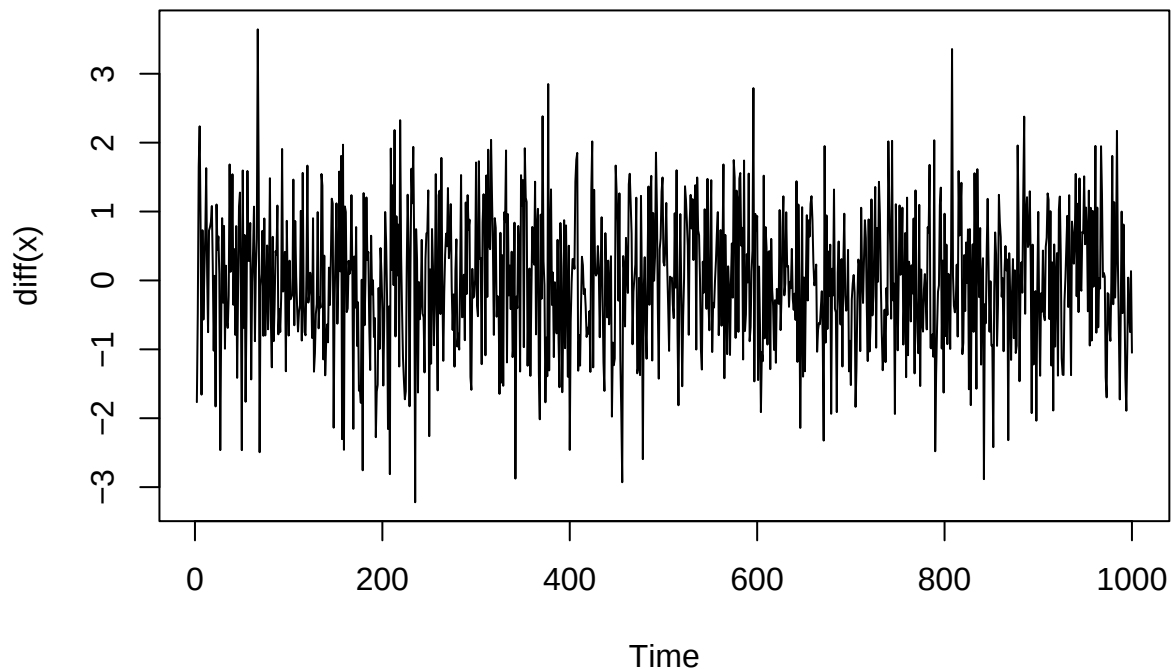
```
res_099_1000 <- check_ar1(phi = 0.99, n = 1000)
```

AR(1) series with $\phi = 0.99$ and $n = 1000$



```
##
## -----
## AR(1) with phi = 0.99 and n = 1000
## -----
##
## Augmented Dickey-Fuller Test
##
## data: x
## Dickey-Fuller = -2.0234, Lag order = 9, p-value = 0.5684
## alternative hypothesis: stationary
```

First differences of AR(1), $\phi = 0.99$, $n = 1000$



```
##  
## ADF test on first differences:  
##  
## Augmented Dickey-Fuller Test  
##  
## data: dx  
## Dickey-Fuller = -10.473, Lag order = 9, p-value = 0.01  
## alternative hypothesis: stationary
```

- **CASE 3:** $n = 10000$, $\phi = 0.99$ (increase n)

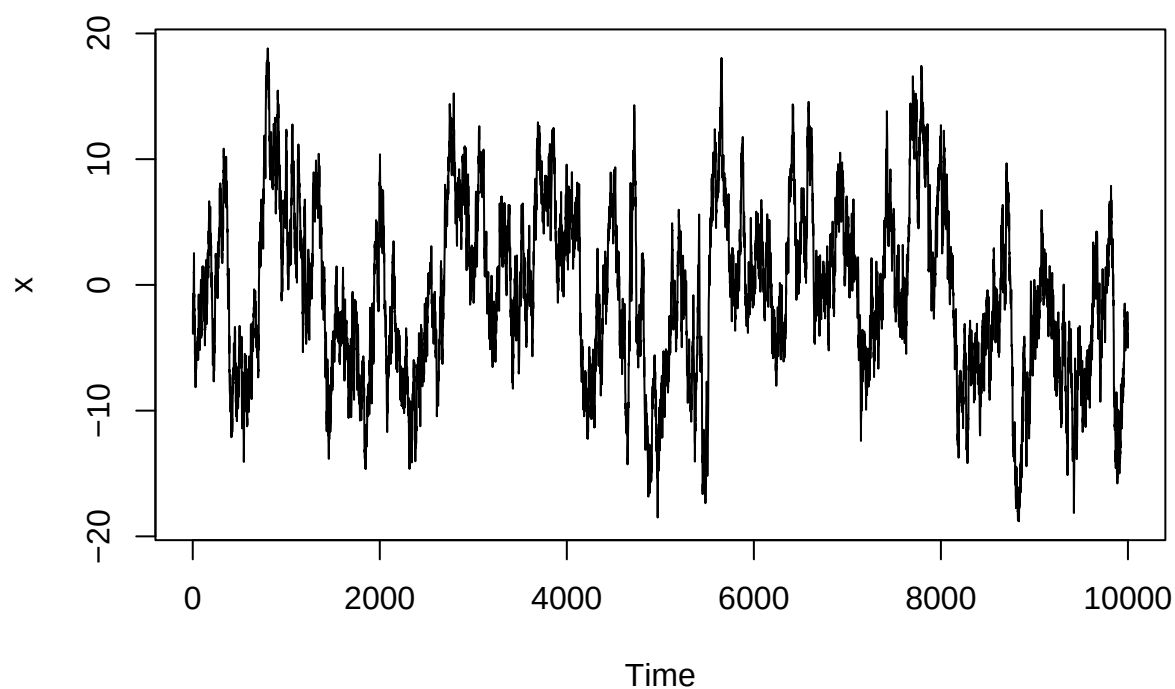
With n increasing, we are giving the ADF test much more information so it should clearly reject the random-walk hypothesis, and in fact the output below shows that the p-value is again very small (0.01). On the visual side as well, the series looks stationary and not a random walk unlike the previous case.

So a larger sample size helps to distinguish better between an AR(1) and true random walk.

The plot of $\text{diff}(x)$ fluctuates around zero very clearly, indicating stationarity.

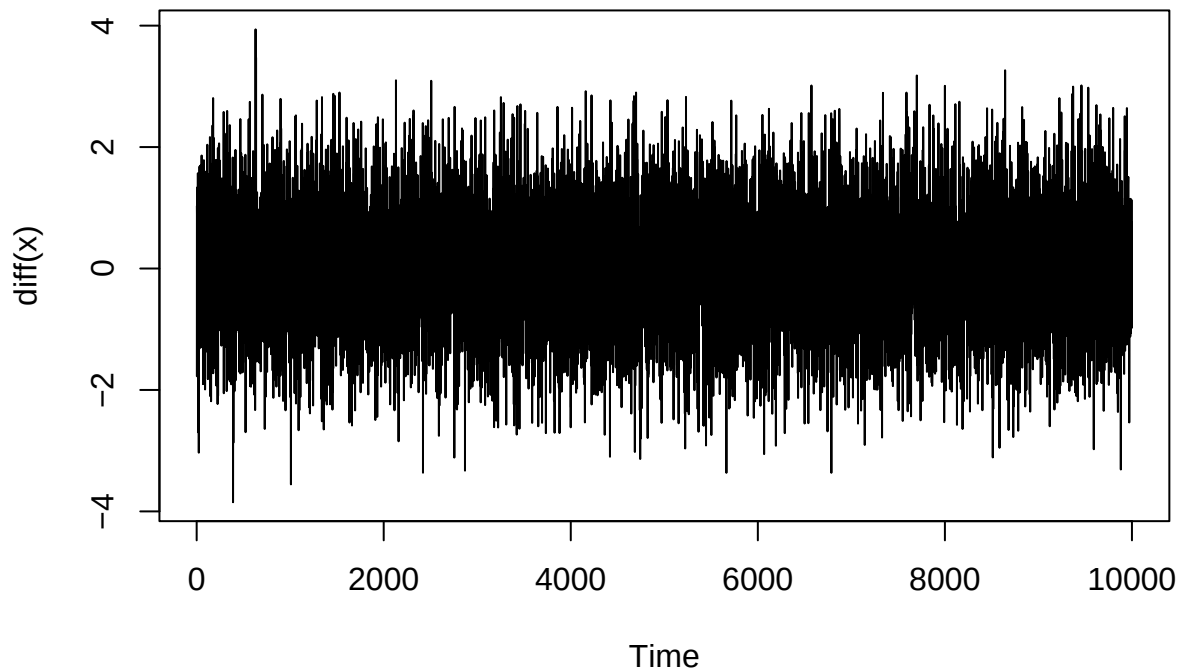
```
res_099_10000 <- check_ar1(phi = 0.99, n = 10000)
```


AR(1) series with $\phi = 0.99$ and $n = 10000$



```
##
## -----
## AR(1) with phi = 0.99 and n = 10000
## -----
##
## Augmented Dickey-Fuller Test
##
## data: x
## Dickey-Fuller = -6.8379, Lag order = 21, p-value = 0.01
## alternative hypothesis: stationary
```

First differences of AR(1), $\phi = 0.99$, $n = 10000$



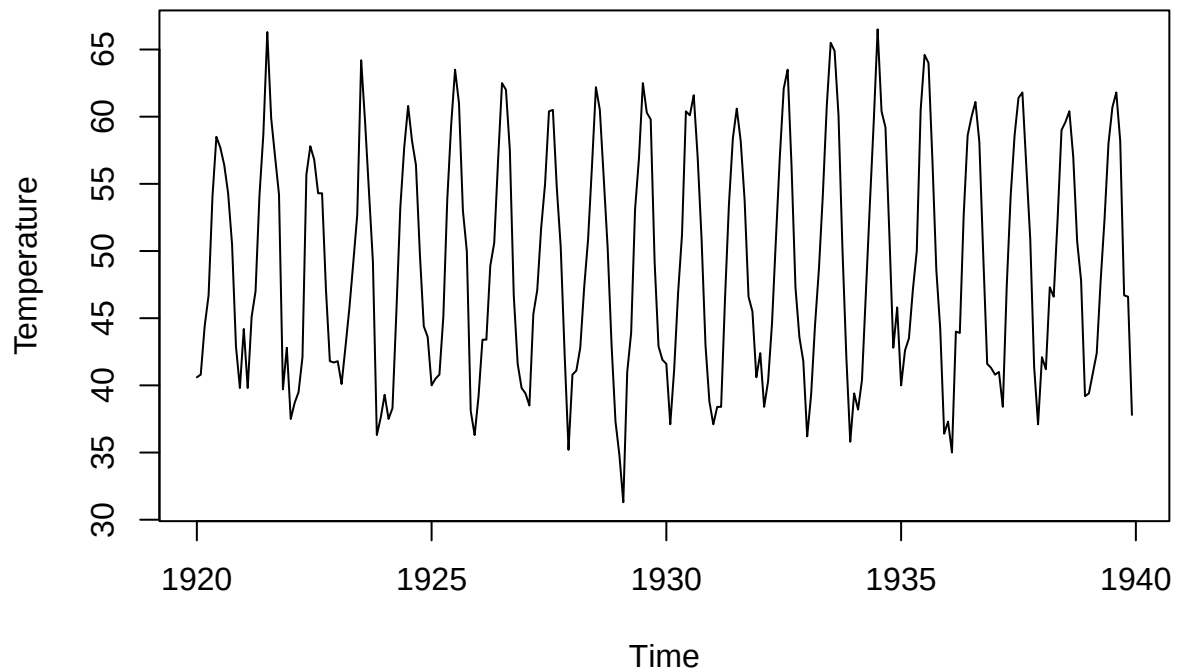
```
##  
## ADF test on first differences:  
##  
## Augmented Dickey-Fuller Test  
##  
## data: dx  
## Dickey-Fuller = -22.586, Lag order = 21, p-value = 0.01  
## alternative hypothesis: stationary
```

B) Fitting SARIMA models:

We are now going to use the *nottem* dataset:

```
library(forecast)  
  
x <- nottem # monthly temperatures in Nottingham  
plot(x, main = "nottem: Monthly Temperatures", ylab = "Temperature", xlab = "Time")
```

nottem: Monthly Temperatures



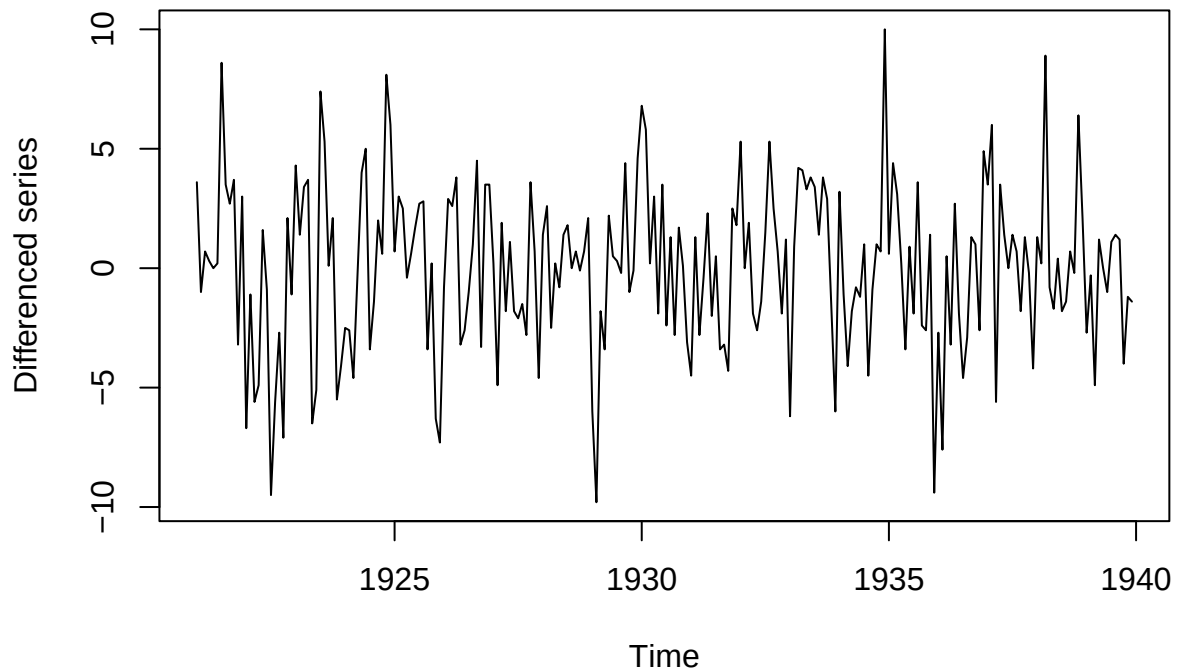
The series shows clear seasonality, so a seasonal model is appropriate.

```
#Seasonal differencing
```

```
y <- diff(x, lag = 12)
```

```
plot(y, main = "Seasonally Differenced Series: diff(nottem, 12)", ylab = "Differenced series", xlab = "Time")
```

Seasonally Differenced Series: `diff(nottem, 12)`

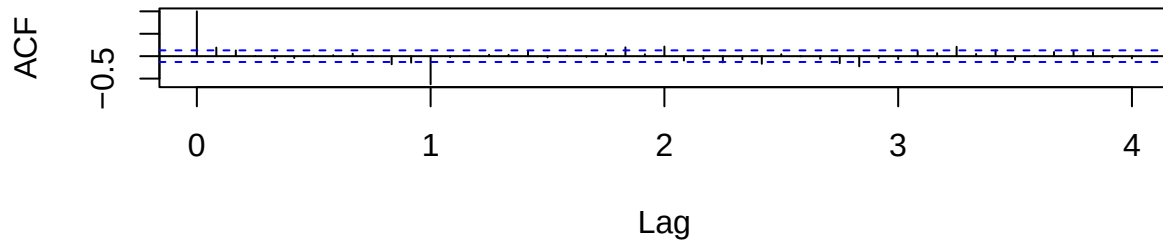


Seasonal differencing at lag 12 removes yearly seasonal structure. Now the new series should look more stationary.

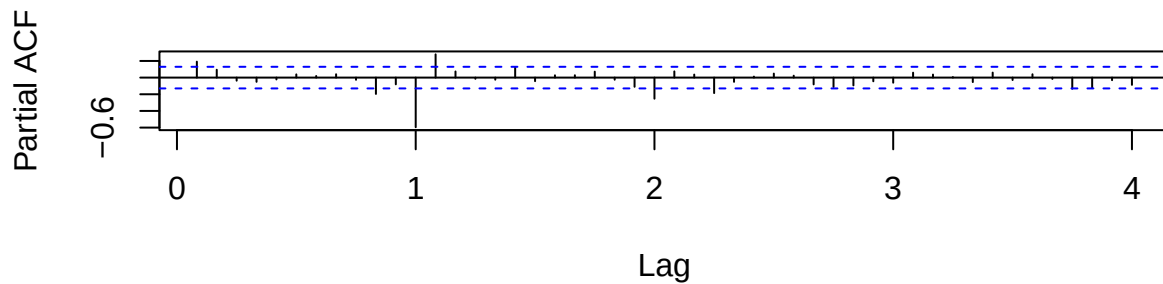
```
# ACF and PACF of the differenced series
```

```
par(mfrow = c(2, 1))  
acf(y, lag.max = 48, main = "ACF of Seasonally Differenced Series")  
pacf(y, lag.max = 48, main = "PACF of Seasonally Differenced Series")
```

ACF of Seasonally Differenced Series



PACF of Seasonally Differenced Series



```
par(mfrow = c(1, 1))
```

Observation:

The ACF and PACF are plotted up to lag 48 to see several periods of the series. Since the data is monthly, the seasonal period is 12, so we check what happens at lags 12, 24, etc. There are still some spikes, which means that even after differencing there is some remaining dependence. This suggests that we need a SARIMA model with seasonal terms.

Fit the model proposed in the practical: SARIMA(1,0,1)(1,0,1)[12] on the differenced series

```
fit1 <- arima(y, order = c(1, 0, 1), seasonal = list(order = c(1, 0, 1), period = 12))
fit1
```

```
##
## Call:
## arima(x = y, order = c(1, 0, 1), seasonal = list(order = c(1, 0, 1), period = 12))
##
## Coefficients:
##          ar1          ma1          sar1          sma1  intercept
##          0.4449   -0.1933   -0.2807   -0.7527           0.0465
## s.e.    0.1831    0.1981    0.0822    0.0785           0.0519
##
## sigma^2 estimated as 5.128:  log likelihood = -517.72,  aic = 1047.43
```

Observations:

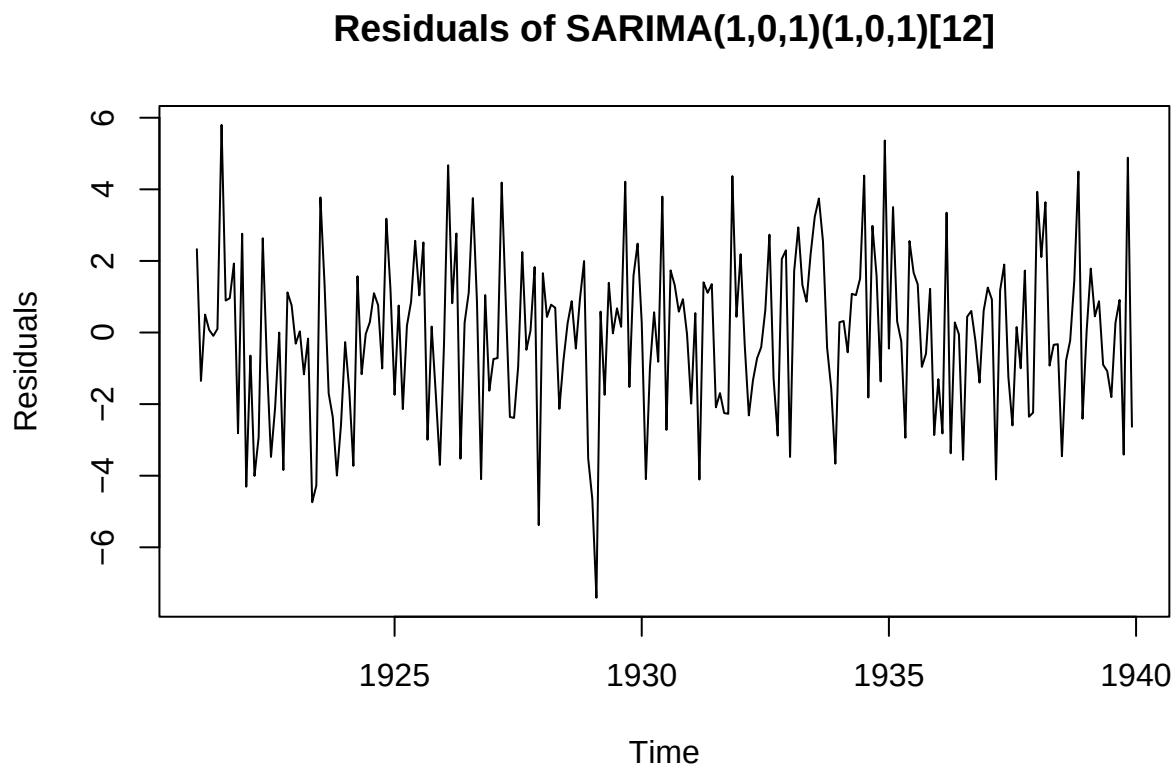
We fitted the model proposed in the practical, a SARIMA(1,0,1)(1,0,1)₁₂ on the seasonally differenced series. The estimated coefficients are used to capture both the short-term dependence and the seasonal dependence remaining in y . The next step is to observe the residuals and see if they behave like white noise, which would indicate whether the model is adequate.

```
fit2=resid(fit1)
fit2
```

##	Jan	Feb	Mar	Apr	May
## 1921	2.328217396	-1.352813290	0.500613502	0.071877976	-0.093485992
## 1922	-4.307470998	-0.643677931	-4.005391399	-2.926562442	2.633571885
## 1923	-0.310696277	0.032464318	-1.167045878	-0.169545340	-4.740815174
## 1924	-0.267388863	-1.600631885	-3.725135968	1.572342416	-1.164966587
## 1925	-1.739564029	0.753693790	-2.138786096	0.192132730	0.857097932
## 1926	-0.334002038	4.673622925	0.817933057	2.766951009	-3.524905910
## 1927	-0.740228278	-0.715364690	4.189470998	0.818458631	-2.360183732
## 1928	1.657537590	0.435277765	0.777785294	0.691895508	-2.133581460
## 1929	-4.652954249	-7.410830707	0.585133213	-1.741322201	1.388947463
## 1930	0.270875529	-4.096500412	-0.970376946	0.566639316	-0.818568889
## 1931	-1.986567348	0.543293238	-4.106490755	1.401918752	1.109599159
## 1932	2.188344263	-0.343302354	-2.315329712	-1.336377327	-0.719522947
## 1933	-3.476372961	1.714427429	2.938438949	1.326912322	0.859755680
## 1934	0.287441708	0.321625635	-0.552928097	1.081475126	1.044990878
## 1935	-0.448119152	3.505153089	0.310644322	-0.250952740	-2.940965112
## 1936	-1.302780709	-2.820090333	3.347359990	-3.373394797	0.282110681
## 1937	1.256259005	0.923535040	-4.104540920	1.178113853	1.898107770
## 1938	3.930608786	2.107754714	3.640451442	-0.923194336	-0.342342967
## 1939	0.040917792	1.784502463	0.451970917	0.873786228	-0.898632410
##	Jun	Jul	Aug	Sep	Oct
## 1921	0.102739587	5.799436126	0.893132018	0.958939523	1.927997175
## 1922	-0.829523747	-3.476673622	-2.094221063	-0.001213856	-3.840988038
## 1923	-4.277218728	3.774460945	1.342043805	-1.699078154	-2.363595749
## 1924	-0.041253156	0.293354309	1.098515761	0.763425162	-1.003914005
## 1925	2.561913404	1.037350317	2.516845656	-2.994080159	0.166142210
## 1926	0.275131986	1.113306270	3.755533651	0.867537367	-4.095014240
## 1927	-2.385400045	-0.956331621	2.248482506	-0.482591117	0.051907949
## 1928	-0.775481298	0.257540815	0.876610093	-0.447055339	0.934938673
## 1929	-0.022485939	0.675151136	0.159588138	4.213287518	-1.521711197
## 1930	3.797225999	-2.720481776	1.734936365	1.325896353	0.583027773
## 1931	1.351740747	-2.089180992	-1.693932157	-2.250193169	-2.270943147
## 1932	-0.408324842	0.632424472	2.733766482	-1.282080247	-2.880808138
## 1933	2.193790533	3.231090322	3.743818855	2.544963607	-0.428530336
## 1934	1.505193344	4.383724935	-1.814384985	2.980358620	1.590058335
## 1935	2.554848097	1.671274988	1.343946455	-0.958353011	-0.594312438
## 1936	-0.051672097	-3.557078399	0.441919429	0.602481515	-0.232914317
## 1937	-1.239330677	-2.596318493	0.152844029	-0.998706435	1.730043376
## 1938	-0.326748978	-3.455135095	-0.785656705	-0.228679101	1.448491092
## 1939	-1.078330744	-1.804954019	0.271834870	0.910129409	-3.410929397
##	Nov	Dec			
## 1921	-2.813558295	2.758884199			
## 1922	1.119806635	0.761594556			
## 1923	-3.997363619	-2.573905862			
## 1924	3.179819703	1.214822788			
## 1925	-1.704146338	-3.700391678			

```
## 1926  1.047588134 -1.626389992
## 1927  1.830780938 -5.380018165
## 1928  1.997115541 -3.517703797
## 1929  1.585408707  2.484046993
## 1930  0.931792284 -0.057249836
## 1931  4.370029812  0.438339540
## 1932  2.054504173  2.296288743
## 1933 -1.561488467 -3.663917980
## 1934 -1.366389678  5.363829301
## 1935  1.220569088 -2.864528620
## 1936 -1.400139620  0.606112574
## 1937 -2.351570960 -2.238659174
## 1938  4.495648502 -2.408066707
## 1939  4.881659680 -2.632003236
```

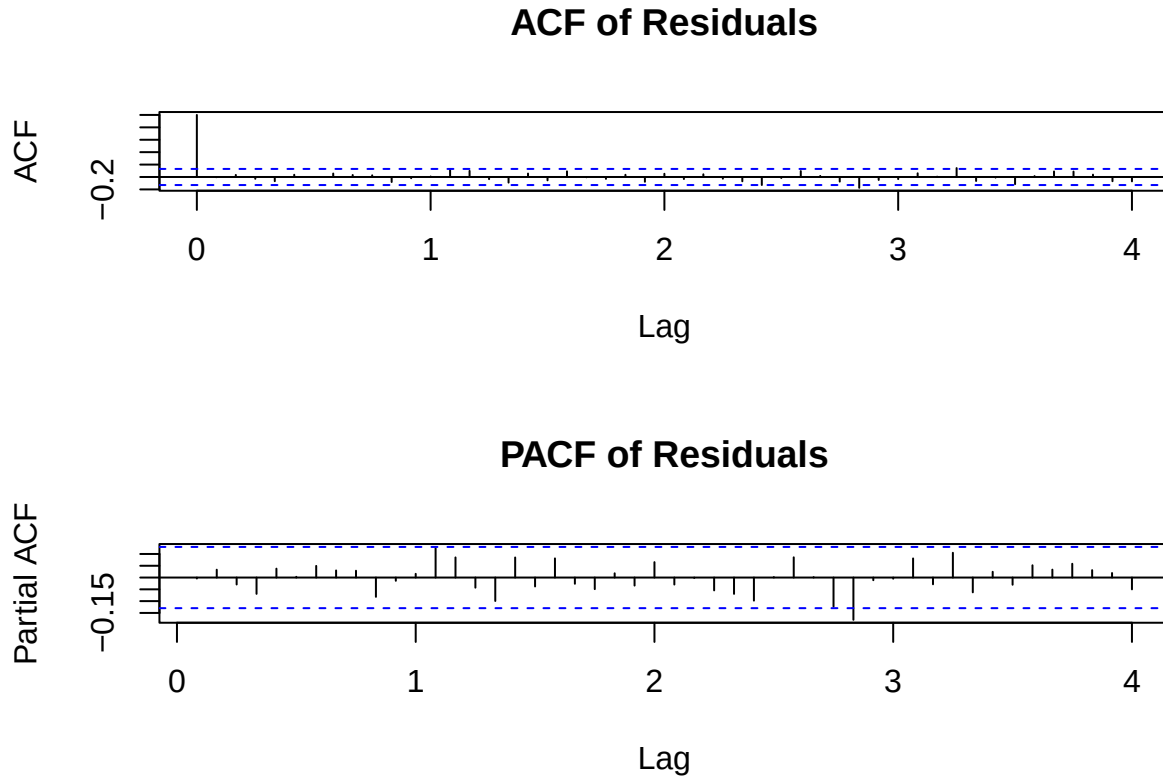
```
plot(fit2, type = "l",
     main = "Residuals of SARIMA(1,0,1)(1,0,1)[12]",
     ylab = "Residuals", xlab = "Time")
```



Observations:

The residuals fluctuate around zero and do not show a clear trend or seasonal pattern. This is great, because for a satisfactory model the residuals should look approximately like white noise. As further check, we will now inspect also the ACF and PACF of the residuals.

```
par(mfrow=c(2,1))
acf(fit2, 48, main = "ACF of Residuals")
pacf(fit2, 48, main = "PACF of Residuals")
```



```
par(mfrow = c(1, 1))
```

Observations:

The ACF and PACF of the residuals do not show strong autocorrelation, so the residual series is close to an IID noise. This suggests that the fitted SARIMA model captures most of the dependence present in the data. If large spikes were still visible, the model would not be adequate and additional terms might be needed.

```
predict(fit1,n.ahead=24)
```

```
## $pred
##           Jan           Feb           Mar           Apr           May           Jun
## 1940  0.55786051 -1.30898845  0.70799643 -1.19260384  0.45587277  1.19746982
## 1941  0.10846201  0.51836138 -0.09851682  0.41236233 -0.06037468 -0.27299381
##           Jul           Aug           Sep           Oct           Nov           Dec
## 1940  1.33389391 -0.09635341 -0.81734310  3.64187280 -2.68020499  1.77519283
## 1941 -0.31327312  0.08728245  0.28925545 -0.96252650  0.81186821 -0.43870171
##
## $se
##           Jan           Feb           Mar           Apr           May           Jun           Jul           Aug
```



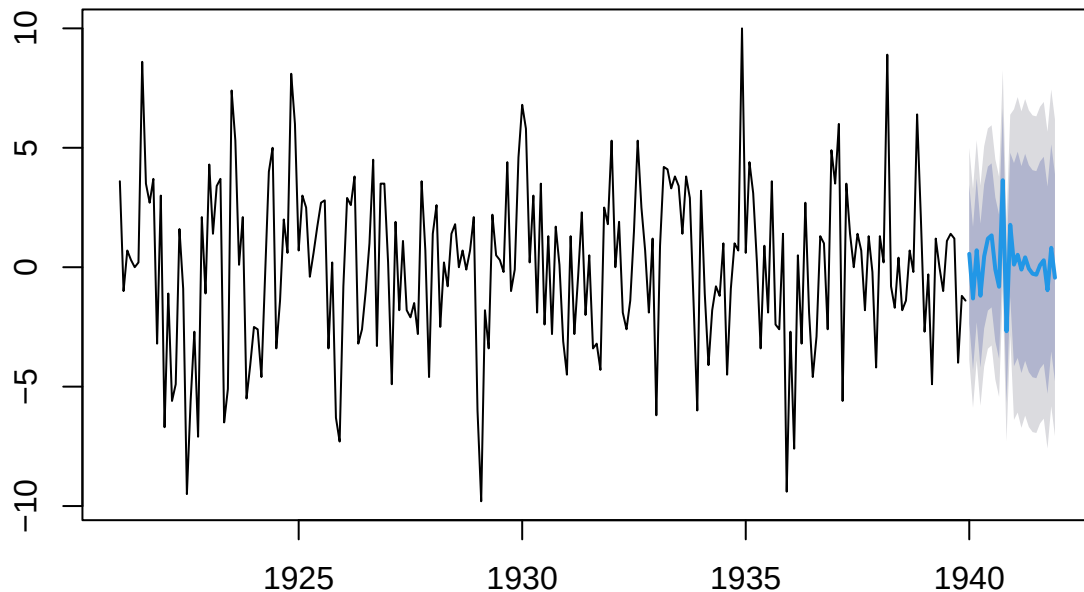
```
## 1940 2.264448 2.335015 2.348729 2.351434 2.351969 2.352075 2.352096 2.352100
## 1941 3.317824 3.369647 3.379809 3.381817 3.382214 3.382293 3.382308 3.382311
##      Sep      Oct      Nov      Dec
## 1940 2.352101 2.352101 2.352101 2.352101
## 1941 3.382312 3.382312 3.382312 3.382312
```

Observations:

The prediction output gives the future estimated values and their associated uncertainty. Since the model was fitted on the seasonally differenced series y , these predictions refer to the differenced process, and not to the original series x . As the forecast horizon increases, the prediction intervals become wider, reflecting increasing uncertainty.

```
library(forecast)
par(mfrow=c(1,1))
fitlfor<-forecast(fit1)
plot(fitlfor)
```

Forecasts from ARIMA(1,0,1)(1,0,1)[12] with non-zero mean



Observations:

The forecast plot shows the future behaviour predicted by the fitted SARIMA model together with prediction intervals:

- The central forecast follows the dependence structure captured by the model
- While the intervals widen as time goes on $\rightarrow$ this is in accordance with what noticed in the previous point, i.e. the uncertainty increases with time.

We now compare simpler seasonal models in order to see whether fewer parameters can still provide residuals close to IID noise. We will try with the following alternatives:

- `fit1_alt`: SARIMA(1,0,0)(1,0,0)[12] → only AR terms
- `fit2_alt`: SARIMA(0,0,1)(0,0,1)[12] → only MA terms
- `fit3_alt`: SARIMA(1,0,1)(0,0,1)[12] → remove seasonal AR

```
# Alternative models (less parameters)

fit1_alt <- arima(y, order=c(1,0,0),
                 seasonal=list(order=c(1,0,0), period=12)
                 )

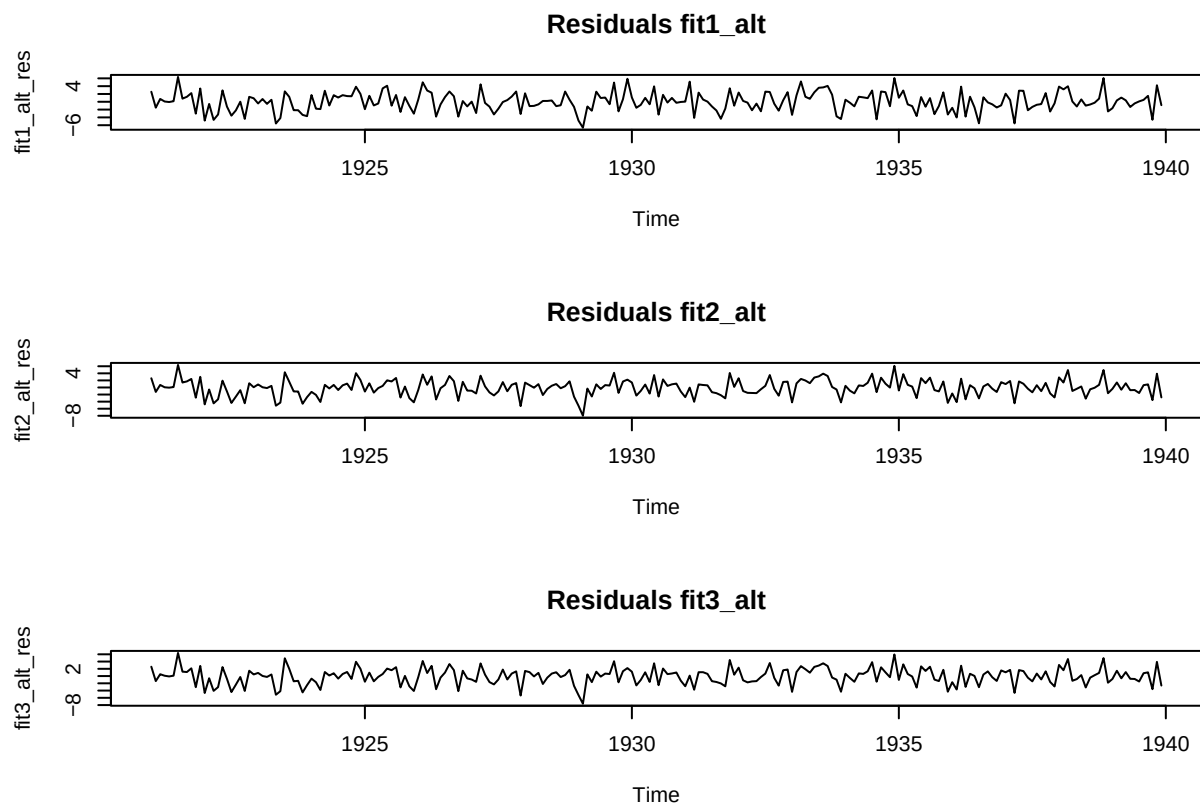
fit2_alt <- arima(y, order=c(0,0,1),
                 seasonal=list(order=c(0,0,1), period=12)
                 )

fit3_alt <- arima(y, order=c(1,0,1),
                 seasonal=list(order=c(0,0,1), period=12)
                 )

# Residuals of the alternative models
fit1_alt_res <- resid(fit1_alt)
fit2_alt_res <- resid(fit2_alt)
fit3_alt_res <- resid(fit3_alt)

par(mfrow=c(3,1))

plot(fit1_alt_res, type="l", main="Residuals fit1_alt")
plot(fit2_alt_res, type="l", main="Residuals fit2_alt")
plot(fit3_alt_res, type="l", main="Residuals fit3_alt")
```



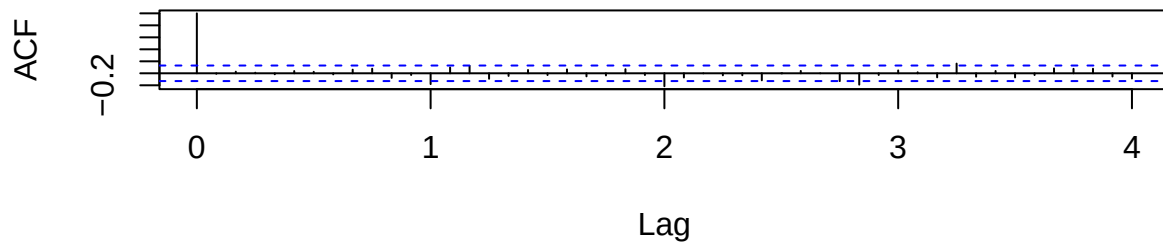
```
par(mfrow=c(1,1))
```

Observations: The residuals of the three alternative models are compared to see whether they behave like white noise. A good model should produce residuals without visible patterns or trends, which is the case for the 3 alternatives.

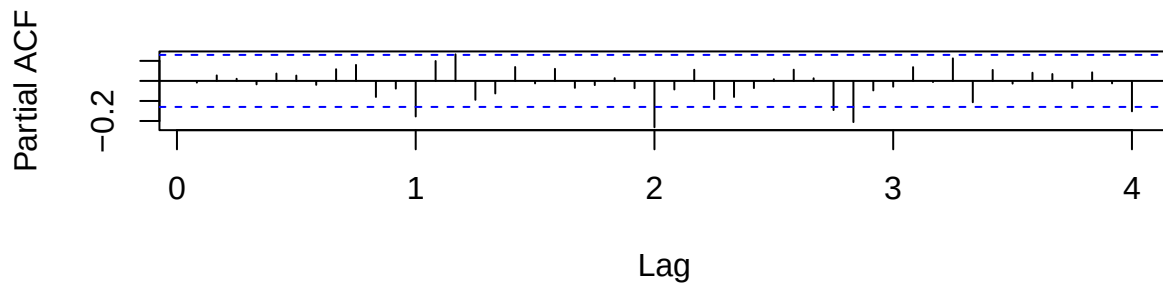
```
# ACF and PACF of residuals
```

```
# fit1_alt  
par(mfrow=c(2,1))  
acf(fit1_alt_res, 48)  
pacf(fit1_alt_res, 48)
```

Series fit1_alt_res

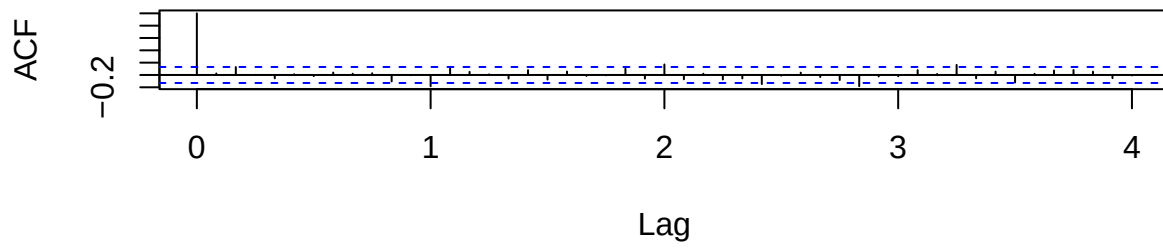


Series fit1_alt_res

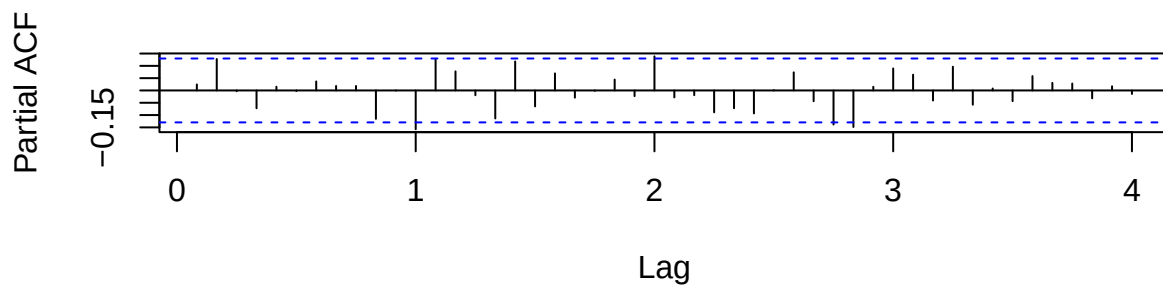


```
# fit2_alt  
par(mfrow=c(2,1))  
acf(fit2_alt_res, 48)  
pacf(fit2_alt_res, 48)
```

Series fit2_alt_res

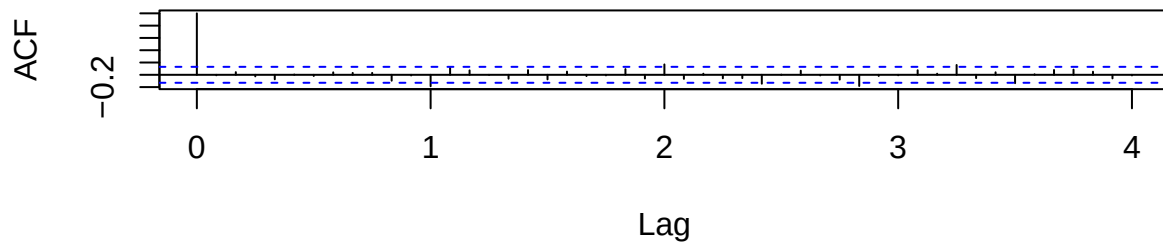


Series fit2_alt_res

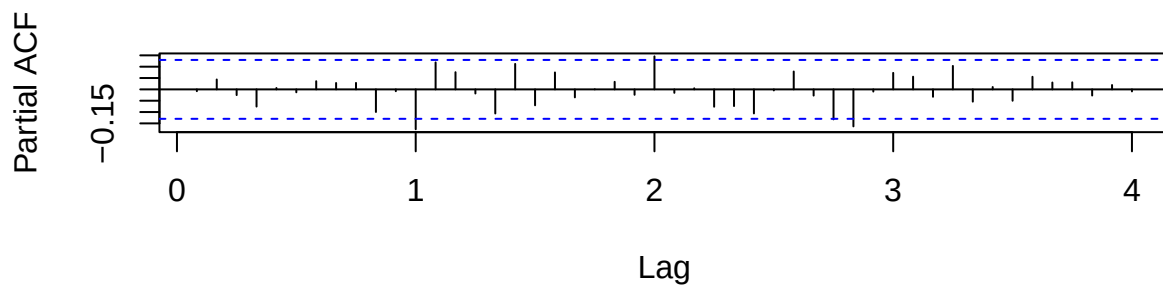


```
# fit3_alt  
par(mfrow=c(2,1))  
acf(fit3_alt_res, 48)  
pacf(fit3_alt_res, 48)
```

Series fit3_alt_res



Series fit3_alt_res



```
par(mfrow=c(1,1))
```

```
# Ljung-Box tests
```

```
Box.test(fit1_alt_res, lag=24, type="Ljung-Box")
```

```
##
## Box-Ljung test
##
## data: fit1_alt_res
## X-squared = 37.638, df = 24, p-value = 0.03775
```

```
Box.test(fit2_alt_res, lag=24, type="Ljung-Box")
```

```
##
## Box-Ljung test
##
## data: fit2_alt_res
## X-squared = 35.203, df = 24, p-value = 0.06545
```

```
Box.test(fit3_alt_res, lag=24, type="Ljung-Box")
```

```
##
## Box-Ljung test
##
## data: fit3_alt_res
## X-squared = 33.111, df = 24, p-value = 0.1018
```

Observations:

We analyze the ACF and PACF of the residuals together with the Ljung-Box test:

- For **fit1_alt**, the Ljung-Box test gives a p-value smaller than 0.05, so we reject the hypothesis of no autocorrelation. This is consistent with the ACF/PACF plots, where some lags exceed the confidence bounds (blue dashed lines), indicating remaining structure in the residuals. Therefore, this model is not adequate.
- For **fit2_alt**, the p-value is slightly above 0.05, so we do not reject the hypothesis, but the result is borderline. The ACF/PACF still show some small peaks, suggesting that some dependence may remain.
- For **fit3_alt**, the p-value is clearly above 0.05, and the ACF/PACF do not show significant spikes outside the confidence bounds. This indicates that the residuals behave like white noise.

Therefore, among the alternative models, **fit3_alt is the best candidate**, since it produces residuals closest to IID noise.

Although the residual plots look similar, the Ljung-Box test provides more clarity in the model comparison.

```
# Prediction
predict(fit1_alt, n.ahead=24)
```

```
## $pred
##           Jan           Feb           Mar           Apr           May
## 1940  1.807920802  0.208553176  3.277652586 -0.791487426  0.009062428
## 1941 -1.196975642 -0.130047426 -2.177424591  0.537073452  0.003031614
##           Jun           Jul           Aug           Sep           Oct
## 1940  0.676166752 -0.724727238 -0.924854537 -0.791435541  2.677452247
## 1941 -0.441989540  0.492538145  0.626041824  0.537038840 -1.777034671
##           Nov           Dec
## 1940  0.809589647  0.943008411
## 1941 -0.530995125 -0.619997954
##
## $se
##           Jan           Feb           Mar           Apr           May           Jun           Jul           Aug
## 1940  2.498270  2.595934  2.603561  2.604168  2.604217  2.604220  2.604221  2.604221
## 1941  3.091837  3.127436  3.130256  3.130481  3.130499  3.130501  3.130501  3.130501
##           Sep           Oct           Nov           Dec
## 1940  2.604221  2.604221  2.604221  2.604221
## 1941  3.130501  3.130501  3.130501  3.130501
```

```
predict(fit2_alt, n.ahead=24)
```

```
## $pred
##           Jan           Feb           Mar           Apr           May           Jun
## 1940  0.14312226 -1.13792897  0.37912199 -0.90120285  0.53012356  0.81903834
```

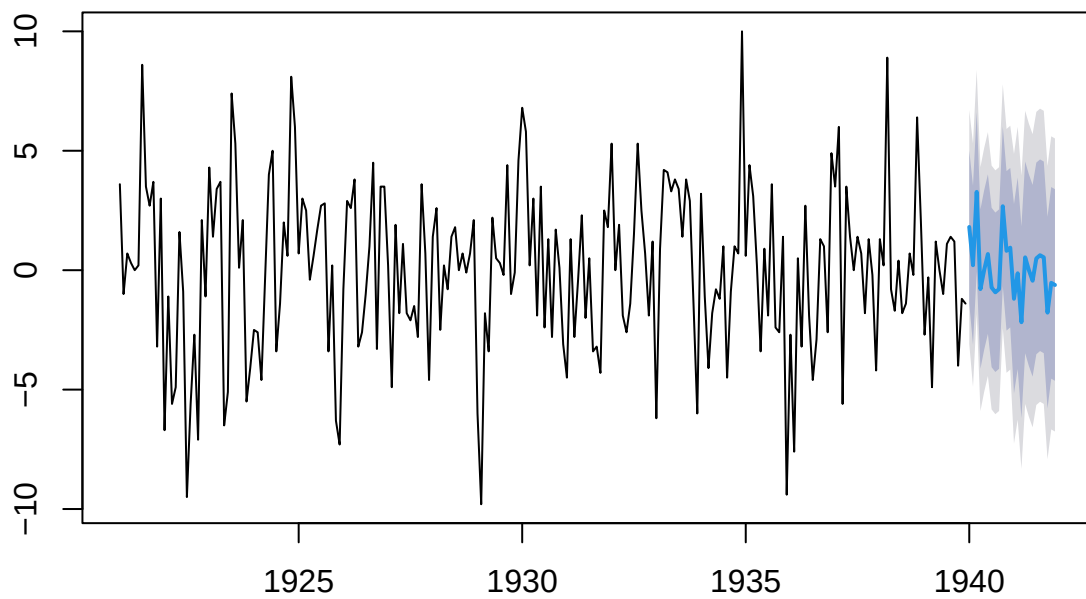
```
## 1941 0.51264339 0.05372554 0.05372554 0.05372554 0.05372554 0.05372554
##      Jul      Aug      Sep      Oct      Nov      Dec
## 1940 1.56265367 -0.27299260 -0.82640721 3.07295394 -2.80776220 1.90318917
## 1941 0.05372554 0.05372554 0.05372554 0.05372554 0.05372554 0.05372554
##
## $se
##      Jan      Feb      Mar      Apr      May      Jun      Jul      Aug
## 1940 2.327358 2.366676 2.366676 2.366676 2.366676 2.366676 2.366676 2.366676
## 1941 3.137075 3.160029 3.160029 3.160029 3.160029 3.160029 3.160029 3.160029
##      Sep      Oct      Nov      Dec
## 1940 2.366676 2.366676 2.366676 2.366662
## 1941 3.160029 3.160029 3.160029 3.160029
```

```
predict(fit3_alt, n.ahead=24)
```

```
## $pred
##      Jan      Feb      Mar      Apr      May      Jun
## 1940 0.33623918 -1.29086811 0.30561534 -0.93821139 0.51029547 0.80624205
## 1941 0.33590466 0.18284094 0.11256849 0.08030599 0.06549409 0.05869386
##      Jul      Aug      Sep      Oct      Nov      Dec
## 1940 1.55435125 -0.28075591 -0.83290390 3.06876883 -2.81183949 1.90240874
## 1941 0.05557183 0.05413849 0.05348044 0.05317832 0.05303961 0.05297593
##
## $se
##      Jan      Feb      Mar      Apr      May      Jun      Jul      Aug
## 1940 2.309859 2.362548 2.373505 2.375808 2.376293 2.376395 2.376416 2.376421
## 1941 3.134473 3.165084 3.171499 3.172849 3.173134 3.173194 3.173206 3.173209
##      Sep      Oct      Nov      Dec
## 1940 2.376422 2.376421 2.376417 2.376398
## 1941 3.173210 3.173210 3.173210 3.173210
```

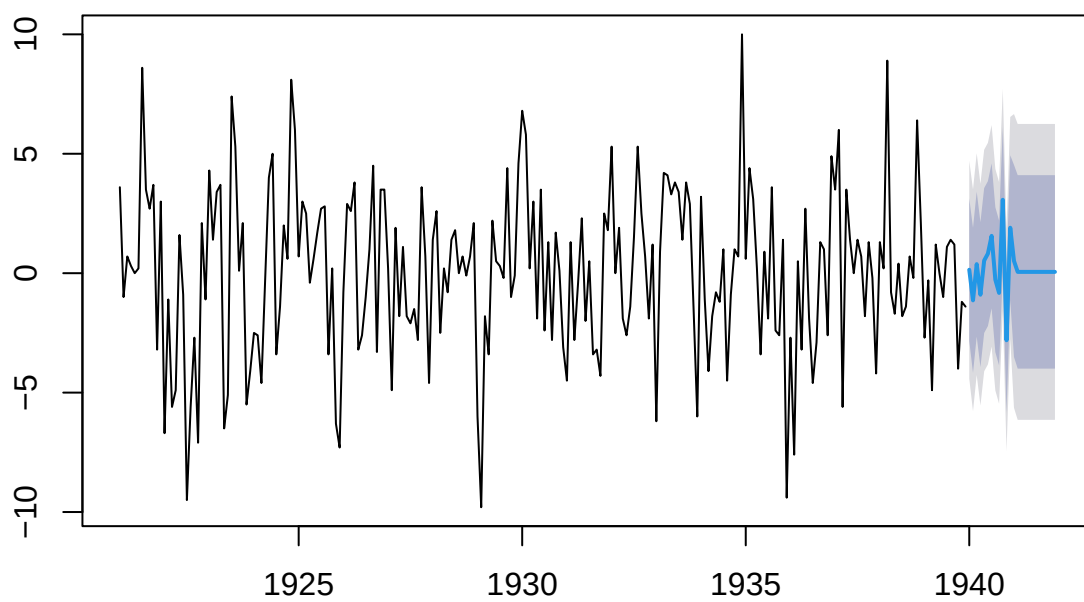
```
#Forecast
fit1_alt_for <- forecast(fit1_alt)
plot(fit1_alt_for)
```


Forecasts from ARIMA(1,0,0)(1,0,0)[12] with non-zero mean



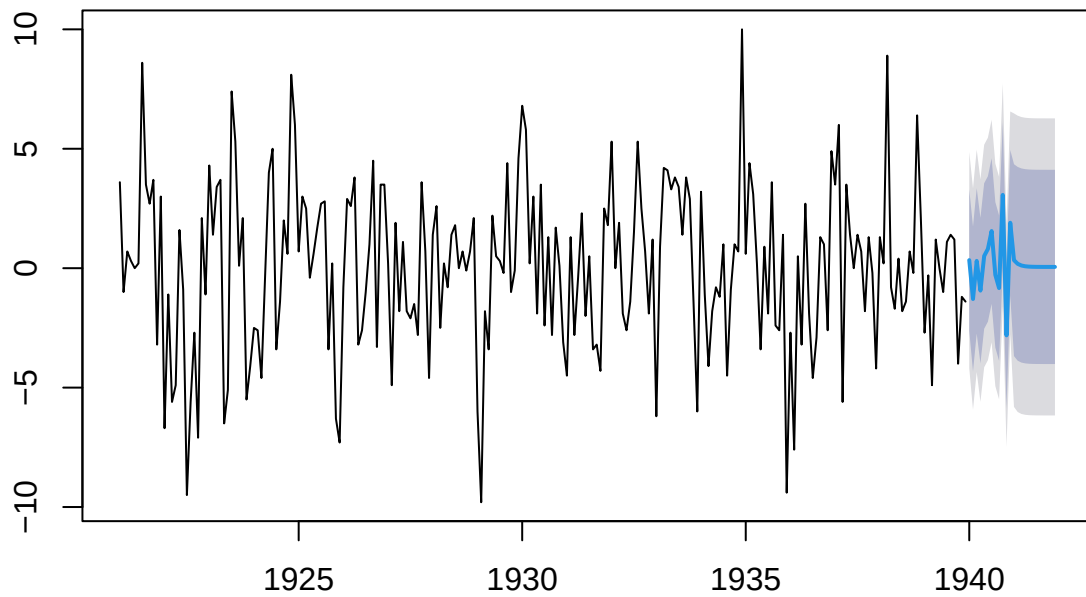
```
fit2_alt_for <- forecast(fit2_alt)
plot(fit2_alt_for)
```

Forecasts from ARIMA(0,0,1)(0,0,1)[12] with non-zero mean



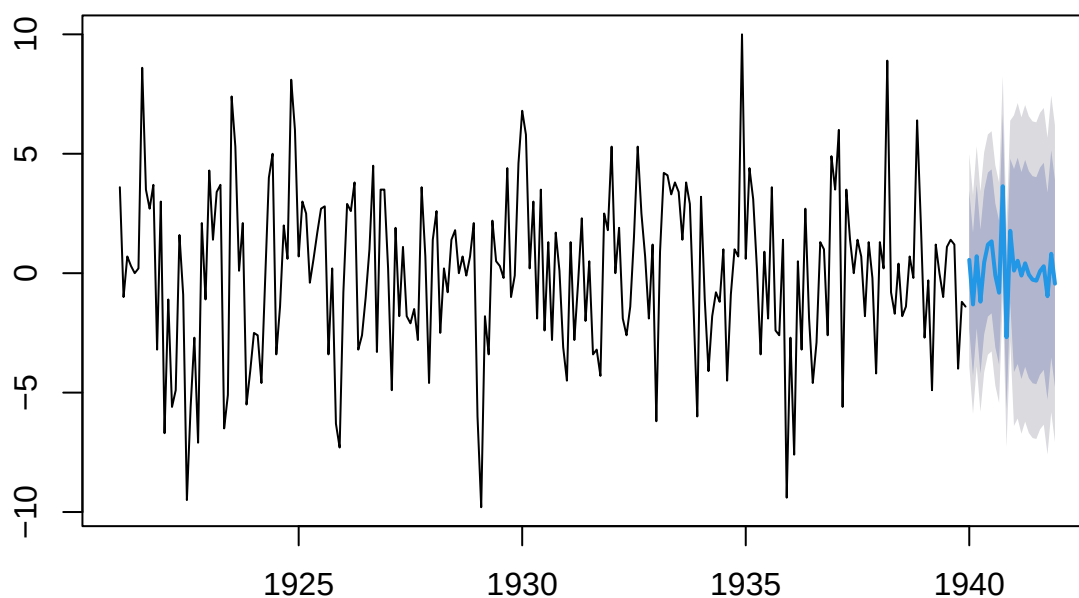
```
fit3_alt_for <- forecast(fit3_alt)
plot(fit3_alt_for)
```

Forecasts from ARIMA(1,0,1)(0,0,1)[12] with non-zero mean



```
# Plotting also the original model forecast for comparison  
fit1_for <- forecast(fit1)  
plot(fit1_for, main="Forecast - main model for comparison")
```

Forecast – main model for comparison



Conclusion:

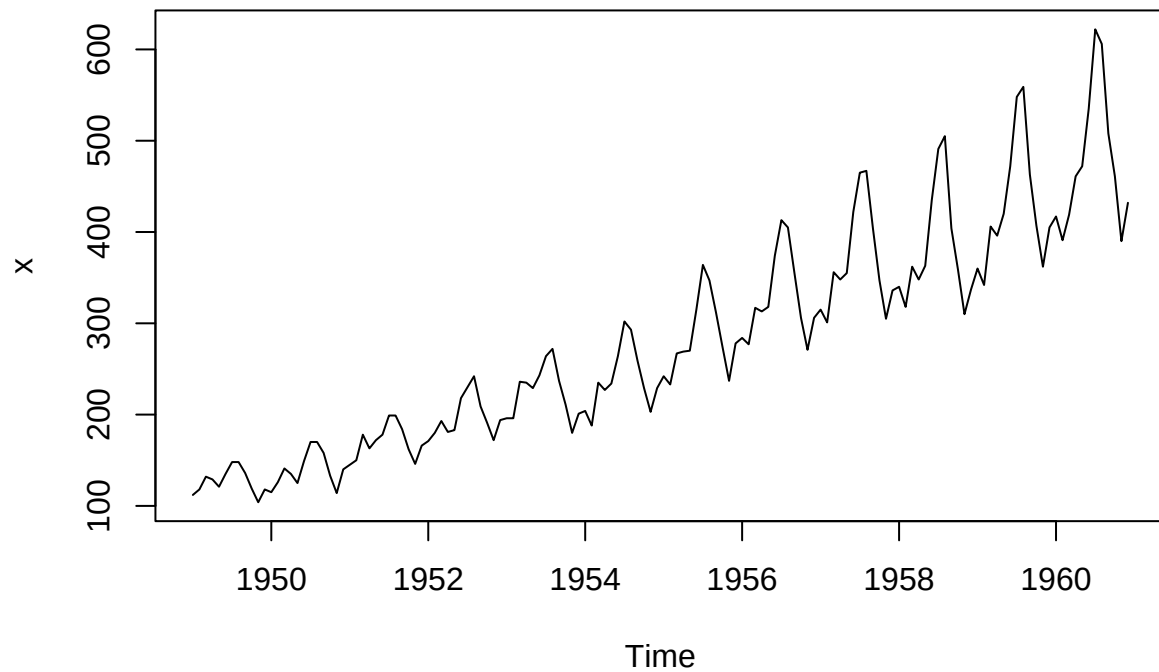
The forecast obtained from `fit1_alt` is noticeably different from the others, showing a different behaviour and larger variability. This is consistent with the residual analysis, where this model was found to be inadequate.

On the other hand, the forecasts from `fit2_alt` and `fit3_alt` are quite similar, suggesting that both models capture the main structure of the series in a comparable way. However, between these two, ***fit3_alt is preferred*** because its residuals are closer to white noise, as confirmed by the Ljung-Box test.

This shows that even if forecasts look similar, the final model selection should be done considering also the residual analysis.

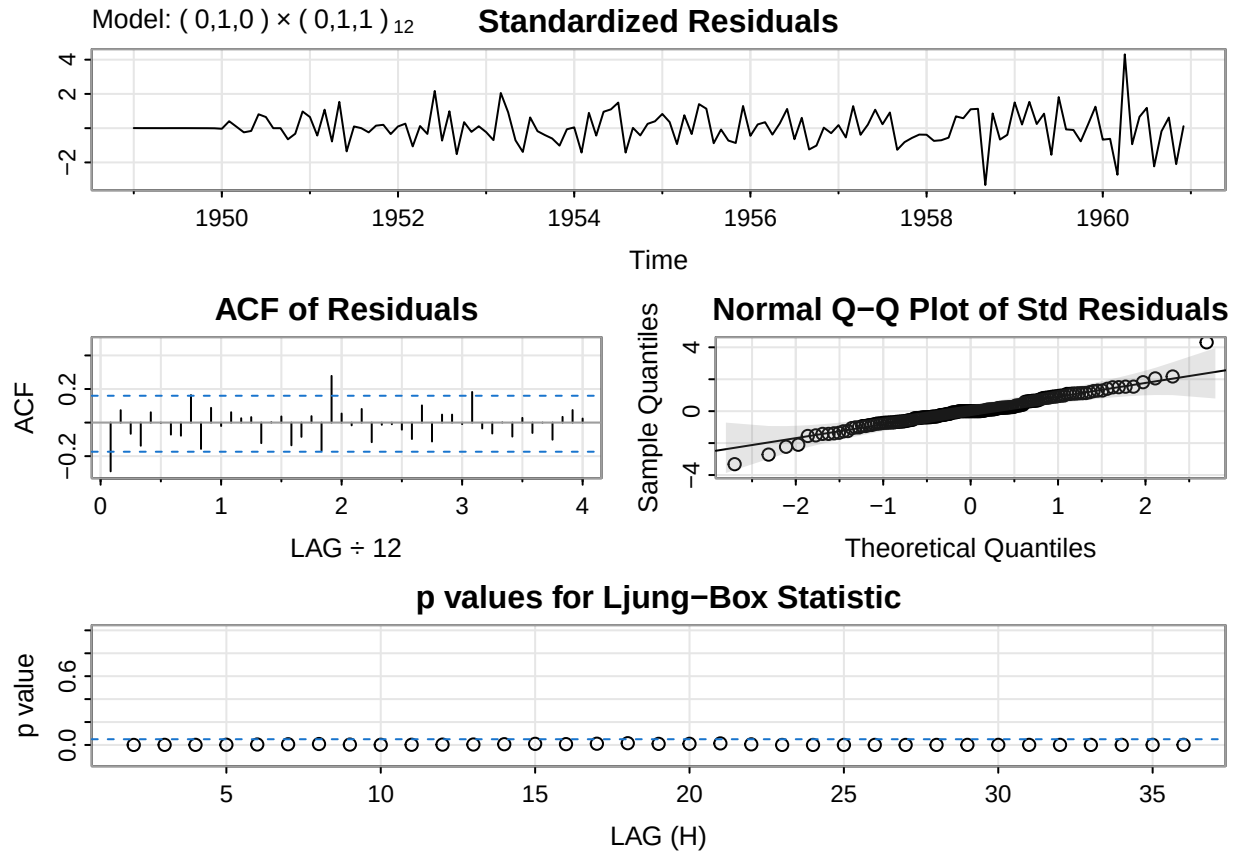
Other example proposed by the practical is with AirPassengers dataset:

```
x<-AirPassengers  
plot(x)
```



```
library(astsa)
fit1_airp<-sarima(x,0,1,0,0,1,0,12)
```

```
## <><><><><><><><><><>
##
## sigma^2 estimated as 151.5573 on 131 degrees of freedom
##
## AIC = 7.874108   AICc = 7.874108   BIC = 7.896056
##
```

Comparison of the models:

We compare the models by analyzing their residuals and diagnostic plots produced by the `sarima` function.

- The **first model**, $\text{SARIMA}(0,1,0)(0,1,0)[12]$, is not adequate, since the residuals show clear autocorrelation and the Ljung-Box test gives small p-values.
- The **second model**, $\text{SARIMA}(1,1,0)(0,1,0)[12]$, improves the fit by including a non-seasonal AR term, but the residuals still show some autocorrelation at higher lags, and the Ljung-Box test is not consistently satisfactory.
- The **third model**, $\text{SARIMA}(0,1,0)(0,1,1)[12]$, provides the **best fit**. The residuals behave closer to white noise, the ACF shows no significant spikes, and the Ljung-Box p-values are consistently high.

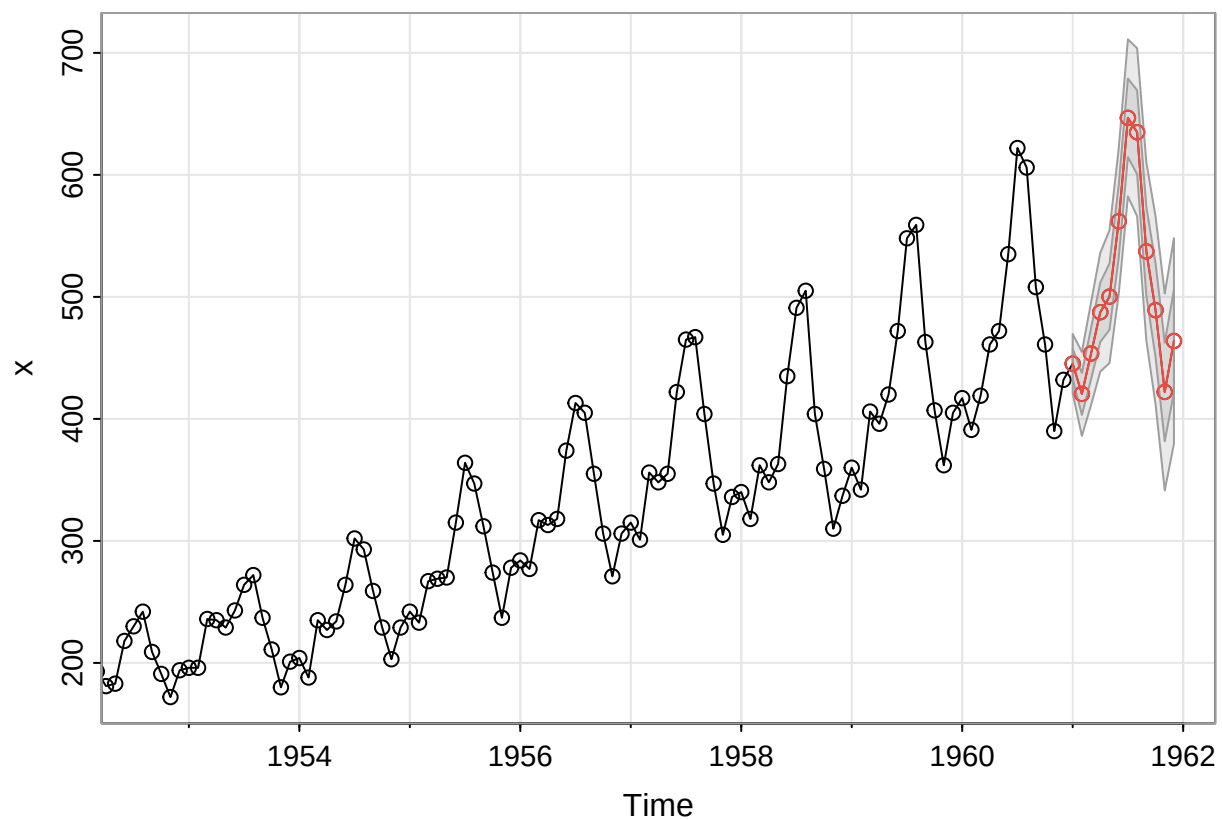
Conclusion:

Among the three models, the **$\text{SARIMA}(0,1,0)(0,1,1)[12]$** model is the best, since it produces residuals closest to IID noise. It successfully captures the seasonal structure of the data while keeping the model relatively simple.

Prediction:

We compute a 12-step ahead forecast using the selected model:

```
# Prediction using the selected SARIMA model
sarima.for(x, 12, 0, 1, 0, 0, 1, 1, 12)
```

```
## $pred
##      Jan      Feb      Mar      Apr      May      Jun      Jul      Aug
## 1961 445.2687 420.4501 453.5973 487.4388 500.2113 561.9579 646.7129 634.9328
##      Sep      Oct      Nov      Dec
## 1961 537.2749 489.1019 422.0791 463.8653
##
## $se
##      Jan      Feb      Mar      Apr      May      Jun      Jul      Aug
## 1961 12.16840 17.20871 21.07628 24.33679 27.20936 29.80636 32.19455 34.41742
##      Sep      Oct      Nov      Dec
## 1961 36.50519 38.47985 40.35800 42.15256
```

The forecast follows the increasing trend and seasonal pattern observed in the data. The prediction intervals widen over time, reflecting increasing uncertainty in the forecasts.